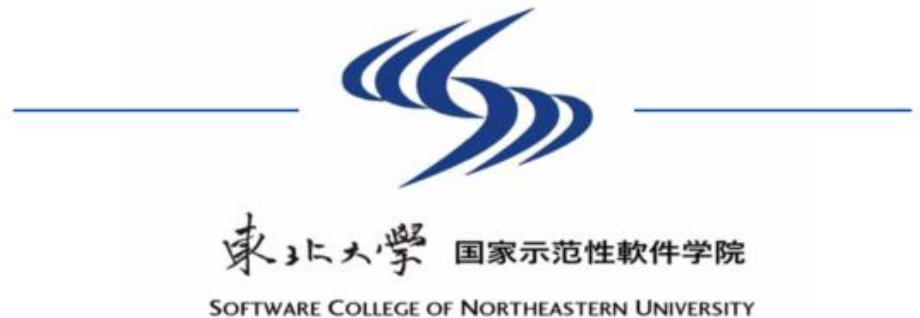


Digital Design and Computer Architecture



考核方式及成绩评定

总分100分，三部分组成

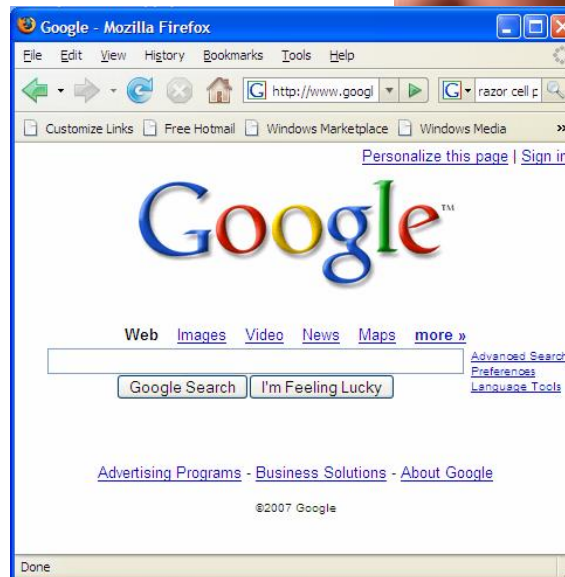
1. 线上成绩：MOOC学习记录，20%（包括：做周测题、做单元测验题、讨论区发声、完成线上期末试题） MOOC
2. 线下成绩：慕课堂10% icourse
3. 实验成绩：20% Experiment
4. 期末考试：50%，闭卷 Final exam

Chapter 1 :: Topics

- **Background**
- **The Game Plan**
- **The Art of Managing Complexity**
- **The Digital Abstraction**
- **Number Systems**
- **Logic Gates**
- **Logic Levels**
- **CMOS Transistors**
- **Power Consumption**

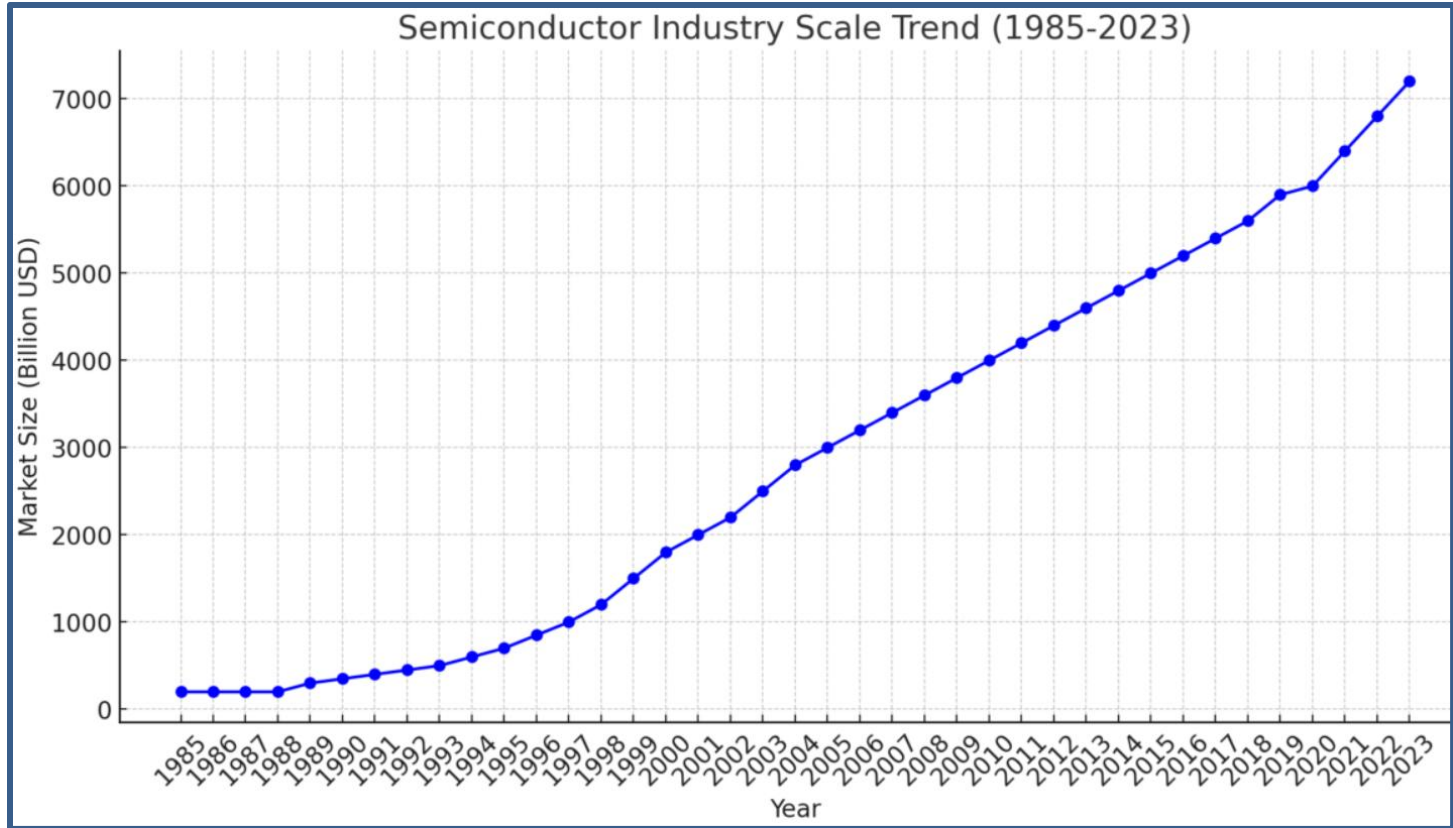
Background

- Microprocessors have revolutionized our world
 - Cell phones, Internet, rapid advances in medicine, etc.



Background

Semiconductor Industry Scale Trend (1985-2023)

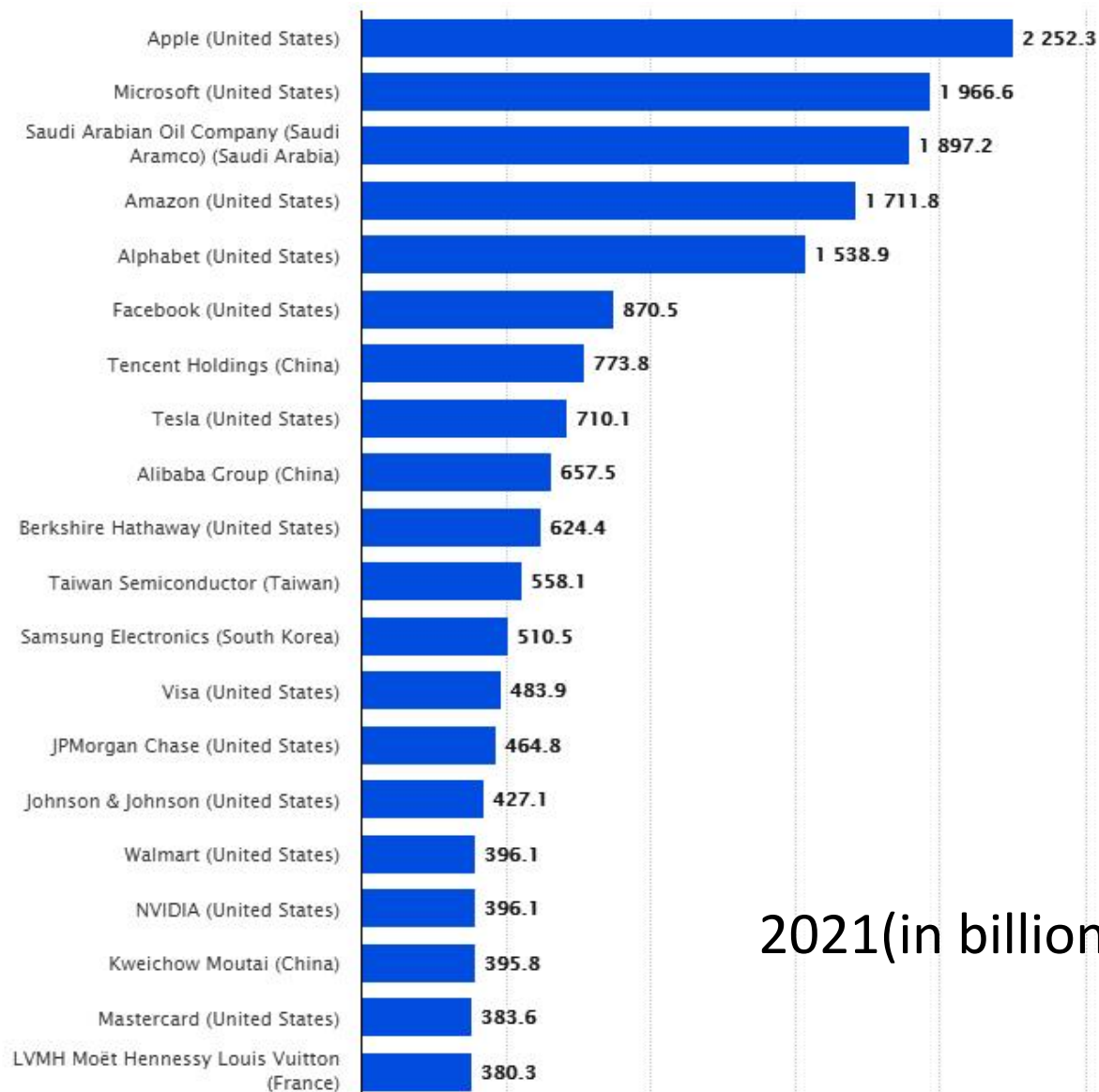


Background

Top companies by market capitalization (as of Dec. 20)

Ranking		Company	Sector	Market cap (in billions of dollars)	Year-to-date change (in percent)
2019	'18				
1	–	Saudi Aramco (Saudi Arabia)	Oil	1,893	–
2	2	Apple (U.S.)	Electronics	1,242	66
3	1	Microsoft (U.S.)	Software	1,201	54
4	4	Alphabet (U.S.)	Internet software/ services	931	29
5	3	Amazon (U.S.)	E-commerce	886	20
6	7	Facebook (U.S.)	Internet software/ services	588	56
7	8	Alibaba (China)	E-commerce	570	60
8	5	Berkshire Hathaway (U.S.)	Conglomerate	554	10
9	6	Tencent (China)	Internet software/ services	459	20
10	10	JPMorgan (U.S.)	Banking	430	33

Background



2021(in billion U.S. dollars)

Background

附：“2022全球市值100强上市公司” 排行榜

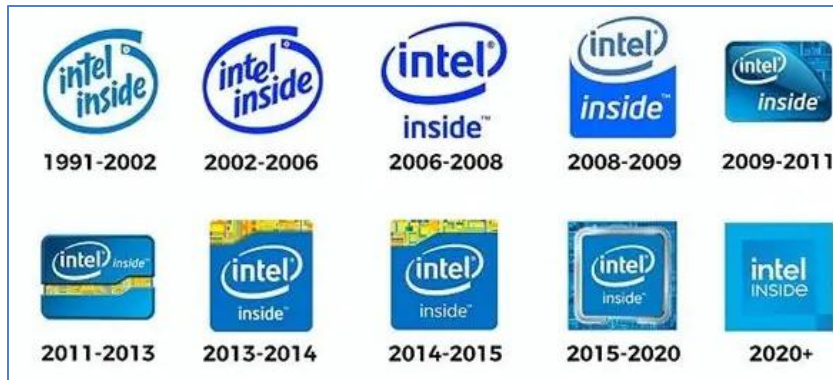
排名	公司	领域	股票市值/上年市值
1	苹果	科技	28500亿美元/20510亿美元
2	微软	科技	23110亿美元/17780亿美元
3	沙特阿美	能源	22980亿美元/19200亿美元
4	ALPHABET INC(谷歌母公司)	科技	18420亿美元/13930亿美元
5	亚马逊	非消费必需品	16590亿美元/15580亿美元
6	特斯拉	非消费必需品	11140亿美元/6410亿美元
7	伯克希尔	金融	7800亿美元/5880亿美元
8	英伟达	科技	6850亿美元/3310亿美元
9	META PLATFORMS(脸书母公司)	科技	6050亿美元/8390亿美元
10	台积电	科技	5410亿美元/5340亿美元
11	维萨	工业	4800亿美元/4680亿美元
12	联合健康	医疗保健	4800亿美元/3520亿美元
13	强生	医疗保健	4660亿美元/4330亿美元
14	腾讯	科技	4590亿美元/7530亿美元
15	沃尔玛	非消费必需品	4100亿美元/3830亿美元
16	摩根大通	金融	4030亿美元/4650亿美元
17	雀巢	消费必需品	3670亿美元/3220亿美元
18	宝洁	消费必需品	3660亿美元/3330亿美元
19	路威酩轩	非消费必需品	3630亿美元/3370亿美元
20	埃克森美孚	能源	3500亿美元/2360亿美元
21	万事达卡	工业	3490亿美元/3540亿美元
22	三星电子	科技	3420亿美元/4310亿美元
23	贵州茅台	消费必需品	3410亿美元/3850亿美元

Rank	Company Name (English)	公司名称 (Chinese)
1	Apple Inc.	苹果公司
2	Microsoft Corporation	微软公司
3	Alphabet Inc.	字母表公司
4	Amazon.com Inc.	亚马逊公司
5	NVIDIA Corporation	英伟达公司
6	Saudi Aramco	沙特阿美
7	Meta Platforms Inc.	Meta平台公司
8	Berkshire Hathaway Inc.	伯克希尔哈撒韦公司
9	TSMC (Taiwan Semiconduc	台积电 (台湾半导体制造公司)
10	Eli Lilly and Company	礼来公司
11	Broadcom Inc.	博通公司
12	Tesla Inc.	特斯拉公司
13	JPMorgan Chase & Co.	摩根大通
14	Walmart Inc.	沃尔玛公司
15	Novo Nordisk	诺和诺德
16	UnitedHealth Group	联合健康集团
17	Exxon Mobil	埃克森美孚
18	Visa Inc.	维萨公司
19	Tencent Holdings Ltd.	腾讯控股有限公司
20	Mastercard Inc.	万事达卡公司

- 1、苹果(APPLE INC)美国信息技术26090亿美元/28500亿美元
- 2、微软(MICROSOFT CORP)美国信息技术21460亿美元/23110亿美元
- 3、沙特阿美(Saudi Arabian Oil Company)沙特能源18930亿美元/22970亿美元
- 4、ALPHABET INC(谷歌母公司)美国通信服务13300亿美元/18420亿美元
- 5、亚马逊(AMAZON.COM INC)美国非必须消费品10580亿美元/16590亿美元
- 6、英伟达(NVIDIA CORP)美国信息技术6850亿美元/6840亿美元
- 7、伯克希尔(Berkshire Hathaway Inc.)美国金融6760亿美元/7800亿美元
- 8、特斯拉(TESLA INC)美国非必须消费品6590亿美元/11140亿美元
- 9、META PLATFORMS(脸书母公司)美国通信服务5500亿美元/6050亿美元
- 10、维萨(Visa Inc)美国金融4640亿美元/4640亿美元
- 11、腾讯(Tencent Holdings Limited)中国通信服务4620亿美元/4560亿美元
- 12、路威酩轩(LVMH)法国非必须消费品4600亿美元/3630亿美元
- 13、台积电(Taiwan Semiconductor Manufacturing Company Limited)中国台湾信息技术4530亿美元/5400亿美元
- 14、埃克森美孚(Exxon Mobil Corporation)美国能源4460亿美元/3500亿美元
- 15、联合健康集团(UnitedHealth Group Incorporated)美国医疗保健4410亿美元/4800亿美元
- 16、强生(Johnson & Johnson)美国医疗保健4050亿美元/4660亿美元
- 17、沃尔玛(WALMART INC)美国日用消费品3980亿美元/4100亿美元
- 18、摩根大通(JPMorgan Chase & Co.)美国金融3840亿美元/4030亿美元
- 19、诺和诺德(Novo Nordisk A/S)丹麦医疗保健3560亿美元/2540亿美元
- 20、宝洁(The Procter & Gamble Company)美国日用消费品3510亿美元/3660亿美元
- 21、万事达卡(Mastercard Incorporated)美国金融3460亿美元/3490亿美元
- 22、三星电子(Samsung Electronics Co., Ltd.)韩国信息技术3270亿美元/3850亿美元
- 23、雀巢(Nestle S.A.)瑞士日用消费品3260亿美元/3600亿美元
- 24、雪佛龙(Chevron Corporation)美国能源3110亿美元/3170亿美元0亿美元
- 25、礼来(Eli Lilly and Company)美国医疗保健3100亿美元/2580亿美元
- 26、家得宝(The Home Depot, Inc.)美国非消费必需品3000亿美元/3090亿美元
- 27、艾伯维(AbbVie Inc.)美国医疗保健2810亿美元/2860亿美元
- 28、默沙东(Merck & Co., Inc., 默克)美国医疗保健2700亿美元/2070亿美元
- 29、可口可乐(The Coca-Cola Company)美国日用消费品2680亿美元/2690亿美元
- 30、阿斯麦(ASML Holding N.V.)荷兰信息技术2680亿美元/2730亿美元
- 31、博通(Broadcom Inc.)美国信息技术2680亿美元/2570亿美元
- 32、阿里巴巴(Alibaba Group Holding Limited)中国非必须消费品2640亿美元/2920亿美元
- 33、百事公司(PepsiCo, Inc.)美国日用消费品2510亿美元/2320亿美元
- 34、甲骨文(Oracle Corporation)美国信息技术2510亿美元/2210亿美元

Background

- 3 problems
 - Why we focus on microprocessors?



Background

- 3 problems



- What is Computer ?

- capture (I/O device, IoT device)
 - store (disk, distributed storage)
 - process (computation, ...video, sound, table)
 - transmit (network, tele..)
 - visualize

- What is the Key problem(s) ?

- faster
 - bigger
 - smarter
 - cheaper

Chapter 1 :: Topics

- Background
- **The Game Plan**
- The Art of Managing Complexity
- The Digital Abstraction
- Number Systems
- Logic Gates
- Logic Levels
- CMOS Transistors
- Power Consumption

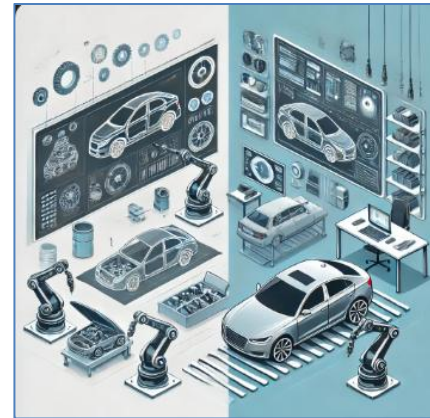
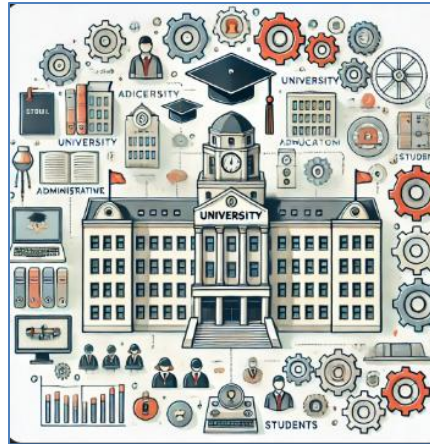
The Game Plan

- Purpose of this course:
 - Understand what's under the hood of a computer or what's inside a computer
 - Learn the principles of digital design
 - Learn to **systematically** debug increasingly complex designs
 - Design and build a microprocessor/CPU

Chapter 1 :: Topics

- Background
- The Game Plan
- **The Art of Managing Complexity**
- The Digital Abstraction
- Number Systems
- Logic Gates
- Logic Levels
- CMOS Transistors
- Power Consumption

The Art of Managing Complexity

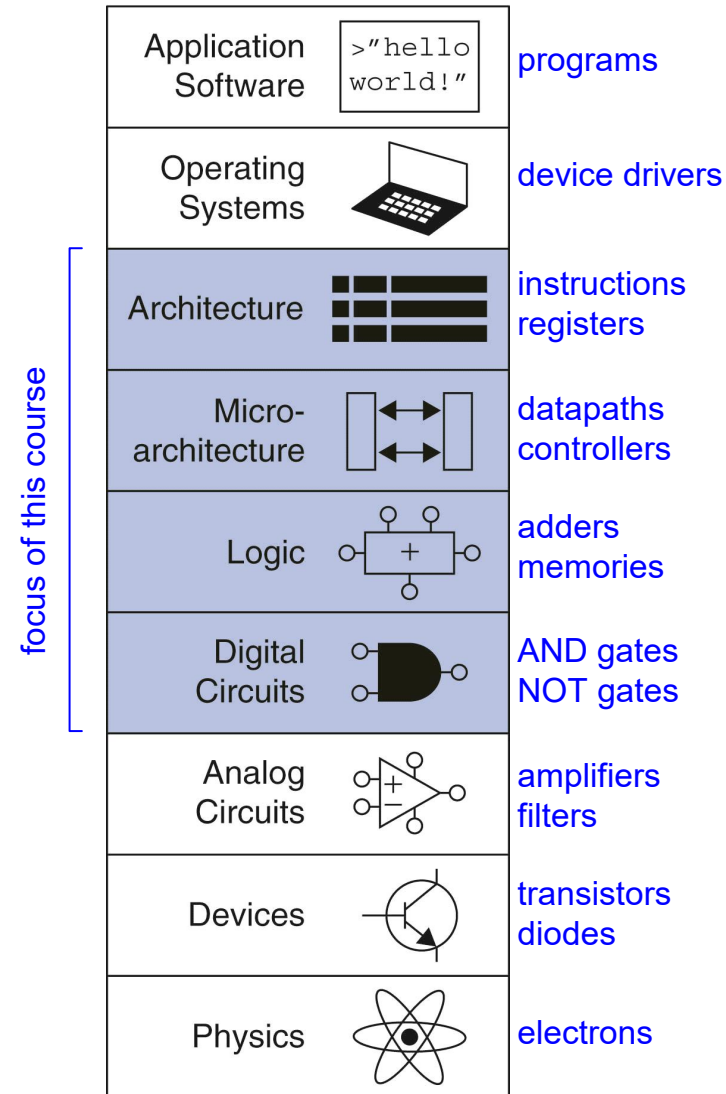


The Art of Managing Complexity

- Abstraction（抽象）
- Discipline（约束）
- The Three –Y's(三原则)
 - Hierarchy
 - Modularity
 - Regularity

Abstraction

- Hiding details when they aren't important
! ! ! !
- let the professionals handle professional issues.



Discipline

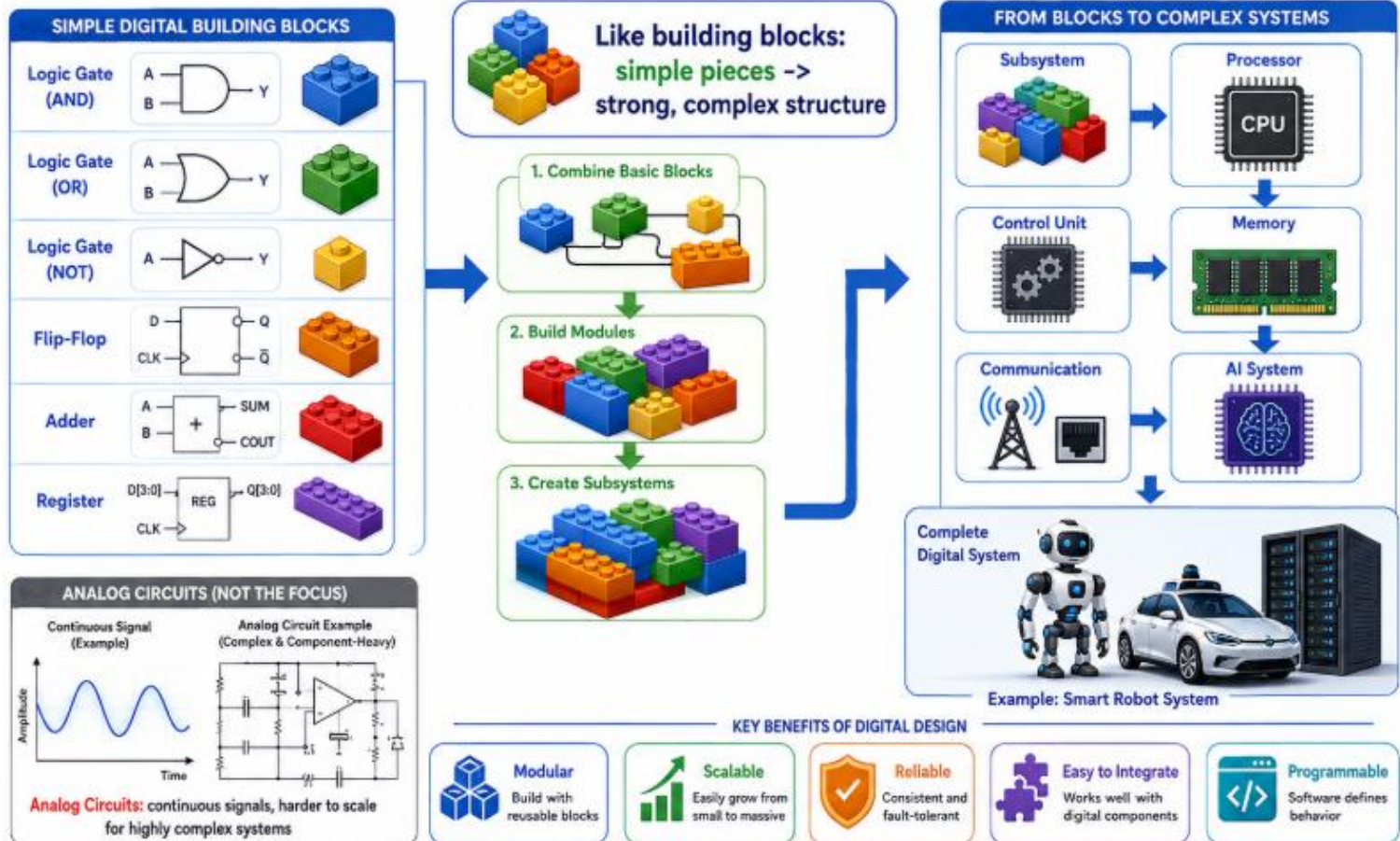
- Intentionally restrict for design choices
- Example: Digital discipline
 - Discrete voltages instead of continuous
 - Simpler to design than analog circuits – can build more **sophisticated** systems
 - Digital systems replacing analog predecessors:
 - i.e., digital cameras, digital television, cell phones, CDs



Discipline

Digital Circuits Build Complex Systems

Simple blocks, powerful combinations



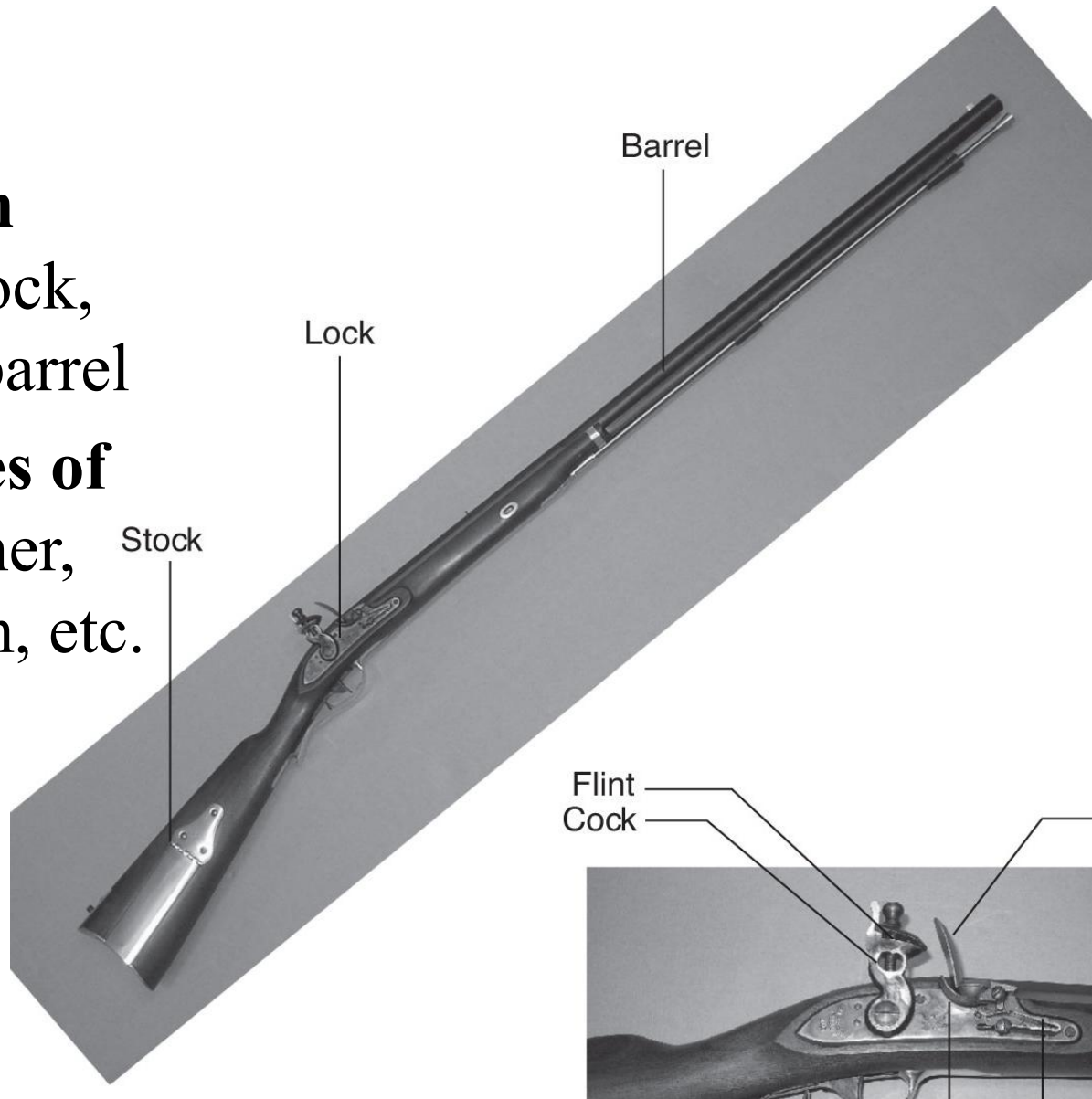
★ Digital design combines simple elements into complex and robust functional systems.

The Three -Y's

- **3 rules to handle complexity of system**
- **Hierarchy (层次化)**
 - A system divided into modules and submodules
- **Modularity (模块化)**
 - Having well-defined functions and interfaces
- **Regularity (规整化)**
 - Encouraging uniformity, so modules can be easily reused

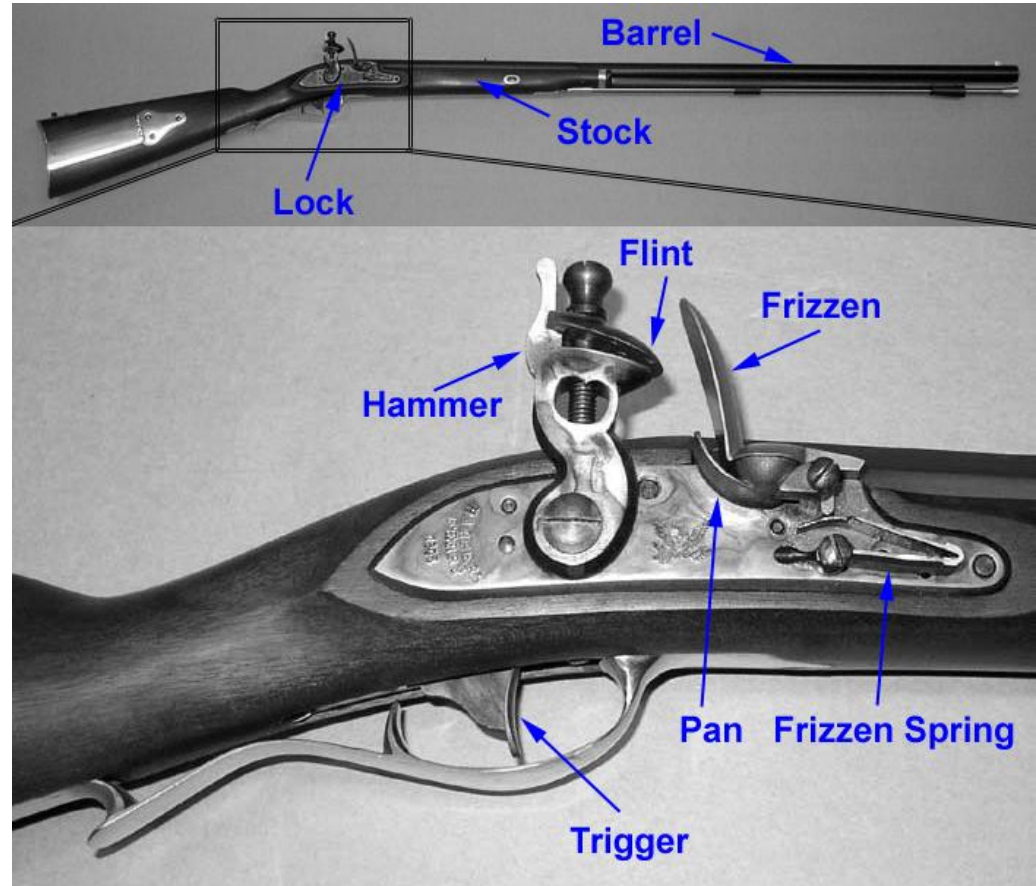
Example: The Flintlock Rifle

- **Hierarchy**
 - **Three main modules:** lock, stock, and barrel
 - **Submodules of lock:** hammer, flint, frizzen, etc.



Example: The Flintlock Rifle

- **Modularity**
 - **Function of stock:** mount barrel and lock
 - **Interface of stock:** length and location of mounting pins
- **Regularity**
 - Interchangeable parts



Chapter 1 :: Topics

- Background
- The Game Plan
- The Art of Managing Complexity
- **The Digital Abstraction**
- Number Systems
- Logic Gates
- Logic Levels
- CMOS Transistors
- Power Consumption

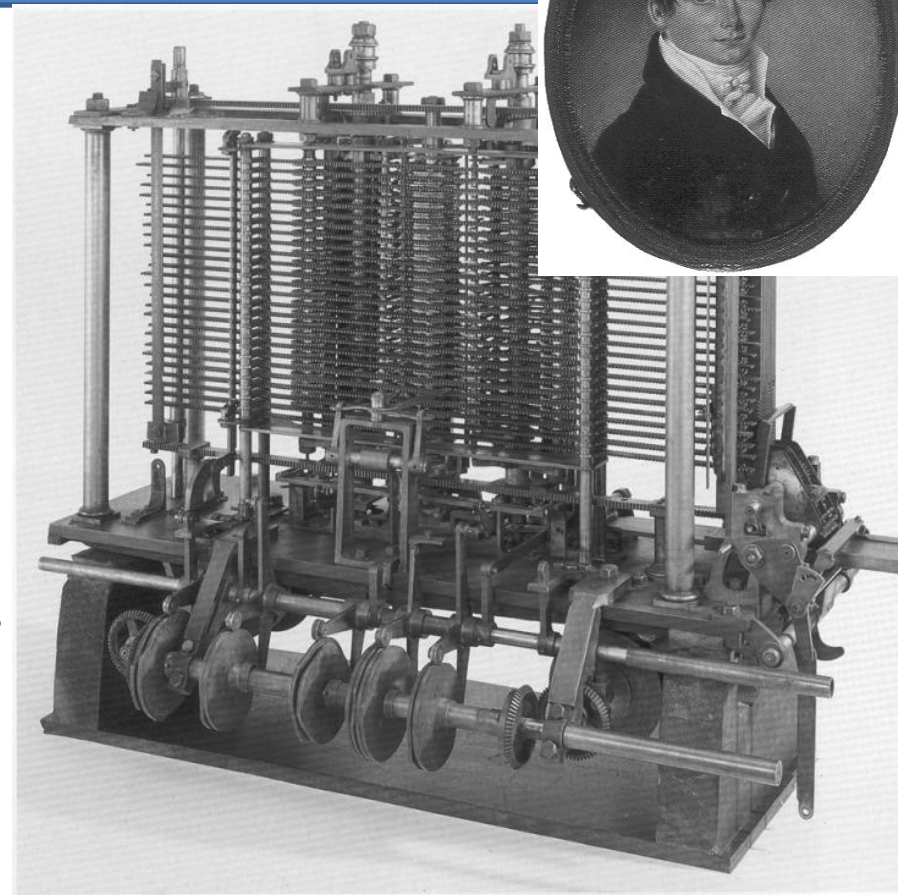
The Digital Abstraction

- Most physical variables are **continuous**
 - Voltage on a wire
 - Frequency of an oscillation
 - Position of a mass
- Digital abstraction considers **discrete subset** of values
- Why we need digital abstraction!!



The Analytical Engine

- Designed by Charles Babbage from 1792 – 1871
- Considered to be the first digital computer
- Built from mechanical gears, where each gear represented a discrete value (0-9)
- Babbage died before it was finished



1812年他协助建立了分析学会，其宗旨是向英国介绍欧洲大陆在数学方面的成就。该学会推动了数学在英国的复兴。

1814年和1817年先后获得文学学士和硕士学位。

1816年，被选为英国皇家学会会员。

1820年，协助创建了剑桥大学天文学会。

1815—1827年期间在伦敦从事科学活动，1827—1828年期间在欧洲大陆考察工厂。

1828—1839年期间在剑桥大学任卢卡斯数学教授（原为艾萨克·牛顿的教席）。

1831年，协助创建了不列颠科学促进协会。

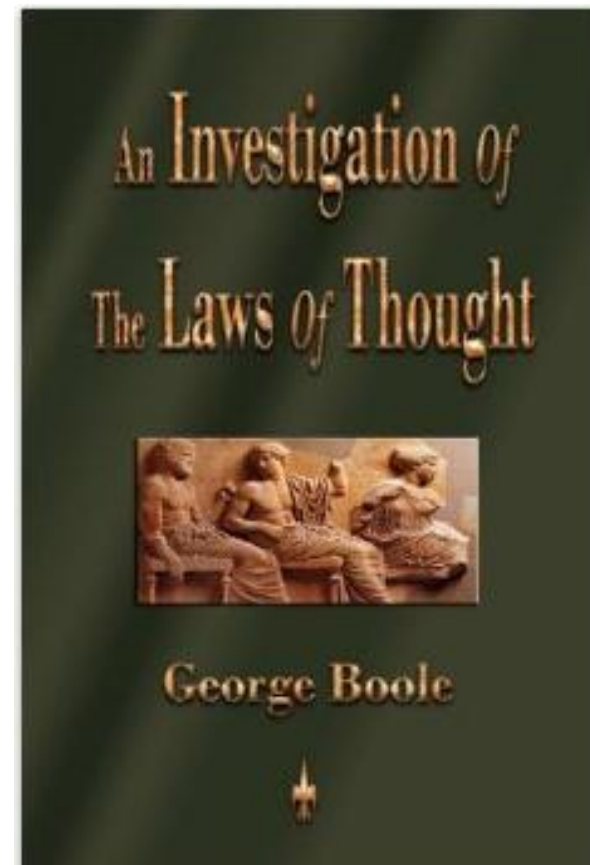
1834年，协助创建了伦敦统计学会。

Digital Discipline: Binary Values

- **Two discrete values:**
 - 1's and 0's
 - 1, TRUE, HIGH
 - 0, FALSE, LOW
- **1 and 0:** voltage levels, rotating gears, fluid levels, etc.
- Digital circuits use **voltage** levels to represent 1 and 0
- ***Bit:*** Binary digit

George Boole, 1815-1864

- Born to working class parents
- Taught himself mathematics and joined the faculty of Queen's College in Ireland.
- Wrote *An Investigation of the Laws of Thought* (1854)
- Introduced binary variables
- Introduced the three fundamental logic operations: AND, OR, and NOT.
- **One of the most important mathematicians in the 19th.**



The Digital Abstraction

Focus on 0 and 1

Ignore the physical meaning (gears, voltage,...)

make low levels abstraction

Chapter 1 :: Topics

- Background
- The Game Plan
- The Art of Managing Complexity
- The Digital Abstraction
- **Number Systems**
- Logic Gates
- Logic Levels
- CMOS Transistors
- Power Consumption

Number Systems

- Decimal numbers
- Binary numbers
- Hexadecimal numbers
- Conversion

Number Systems

Weighted positional number system(带权的位置数制)

A number is represented by a number of digits
each of which evaluates to a prespecified value
the sum of the values contributed by all the digits equals
the value of the number.

Base (基数)

weighted (权)

decimal system

binary system

octal system

hexadecimal system

Number Systems

- Decimal numbers

1's column
10's column
100's column
1000's column

Base-10 number system uses 10 notations for the digits(0~9)
weight: 10^i

$$5374_{10} =$$

1's column
10's column
100's column
1000's column

$$5374_{10} = 5 \times 10^3 + 3 \times 10^2 + 7 \times 10^1 + 4 \times 10^0$$

five three seven four
thousands hundreds tens ones

Number Systems

- Binary numbers

1's column
2's column
4's column
8's column

$$1101_2 =$$

Base-2 number system uses 2 notations for the digits(0~1)
weight: 2^i

1's column
2's column
4's column
8's column

$$1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13_{10}$$

one eight one four no two one one

Powers of Two

- $2^0 =$

- $2^1 =$

- $2^2 =$

- $2^3 =$

- $2^4 =$

- $2^5 =$

- $2^6 =$

- $2^7 =$

- $2^8 =$

- $2^9 =$

- $2^{10} =$

- $2^{11} =$

- $2^{12} =$

- $2^{13} =$

- $2^{14} =$

- $2^{15} =$

Powers of Two

- $2^0 = 1$
- $2^1 = 2$
- $2^2 = 4$
- $2^3 = 8$
- $2^4 = 16$
- $2^5 = 32$
- $2^6 = 64$
- $2^7 = 128$
- $2^8 = 256$
- $2^9 = 512$
- $2^{10} = 1024$
- $2^{11} = 2048$
- $2^{12} = 4096$
- $2^{13} = 8192$
- $2^{14} = 16384$
- $2^{15} = 32768$
- Hard to memorize up to 2^9

Number Systems

- Decimal numbers

10^{-2} 's column
 10^{-1} 's column
1's column
 10 's column

Base-10 number system uses 10 notations for the digits(0~9)
weight: 10^i

$$53.74_{10} =$$

10^{-2} 's column
 10^{-1} 's column
1's column
 10 's column

$$53.74_{10} = 5 \times 10^1 + 3 \times 10^0 + 7 \times 10^{-1} + 4 \times 10^{-2}$$

five three seven four
tens ones tenth hundredth

Number Systems

- Binary numbers

2^{-2} 's column
 2^{-1} 's column
1's column
 2^1 's column

$$11.01_2 =$$

Base-2 number system uses 2 notations for the digits (0~1)
weight: 2^i

2^{-2} 's column
 2^{-1} 's column
1's column
 2^1 's column

$$11.01_2 = 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 3.25_{10}$$

one twoone oneno halfone fourth

Binary Values and Range

- N -digit decimal number
 - How many values?
 - Range?
 - Example: 3-digit decimal number:
- N -bit binary number
 - How many values?
 - Range:
 - Example: 3-digit binary number:

Binary Values and Range



- N -digit decimal number
 - How many values? 10^N
 - Range? $[0, 10^N - 1]$
 - Example: 3-digit decimal number:
 - $10^3 = 1000$ possible values
 - Range: $[0, 999]$
- N -bit binary number
 - How many values? 2^N
 - Range: $[0, 2^N - 1]$
 - Example: 3-digit binary number:
 - $2^3 = 8$ possible values
 - Range: $[0, 7] = [000_2 \text{ to } 111_2]$

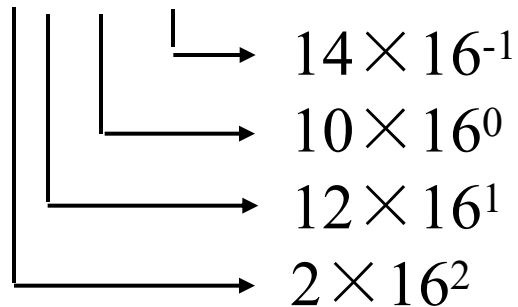
Hexadecimal Numbers

Hex Digit	Decimal Equivalent	Binary Equivalent
0	0	
1	1	
2	2	
3	3	
4	4	
5	5	
6	6	
7	7	
8	8	
9	9	
A	10	
B	11	
C	12	
D	13	
E	14	
F	15	

Hexadecimal Numbers



- Shorthand for binary
- Base -16 number system uses 16 notations for the digits(0~15), 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
- weight: 16^i
- ex: 2CA.E



$(2C7.1F)_{16}$ is expressed as:

$$(2C7.1F)_{16} = 2 \times 16^2 + 12 \times 16^1 + 7 \times 16^0 + 1 \times 16^{-1} + 15 \times 16^{-2}$$

Hexadecimal Numbers

Hex Digit	Decimal Equivalent	Binary Equivalent
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Hexadecimal Numbers

Different number systems for different purposes.

Binary (Base 2)

Uses only two digits: 0 and 1.

Used by computers.

Digits

0 1

Example

11010



1 1 0 1 0

1 = ON 0 = OFF



The language computers understand.

Decimal (Base 10)

Uses ten digits: 0–9.

Used in everyday life.

Digits

0 1 2 3 4
5 6 7 8 9

Example

26



20 apples + 6 bananas = 26



The number system we use every day.

Hexadecimal (Base 16)

Uses sixteen digits: 0–9 and A–F.

A compact shorthand for binary in computing.

Digits

0 1 2 3 4 5 6 7 8 9
A B C D E F

Example

1A



1A (Hex) is a shorter way to write the binary number 11010 (Binary)



Compact, efficient, and widely used in programming.

★ Conversion Example

Decimal

26

=

Binary

11010

=

Hexadecimal

1A

26 in decimal is 11010 in binary.

11010 in binary is 1A in hex.



Key Takeaway: Different systems, same value. Choose the right one for the job!

Number Conversion

- Binary to decimal conversion:
 - Convert 10011_2 to decimal
- Decimal to binary conversion:
 - Convert 47_{10} to binary

Number Conversion

- Binary to decimal conversion:
 - Convert 10011_2 to decimal
 - $16 \times 1 + 8 \times 0 + 4 \times 0 + 2 \times 1 + 1 \times 1 = 19_{10}$
- Decimal to binary conversion:
 - Convert 47_{10} to binary
 - $32 \times 1 + 16 \times 0 + 8 \times 1 + 4 \times 1 + 2 \times 1 + 1 \times 1 = 101111_2$

Conversion between numbers of different representations

- Convert $(1101.01)_2$ to their decimal format

$$\begin{aligned}(1101.01)_2 &= 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} \\ &= 8 + 4 + 1 + 0.25 = 13.25\end{aligned}$$

Hexadecimal to Binary Conversion

- Hexadecimal to binary conversion:
 - Convert $4AF_{16}$ (also written $0x4AF$) to binary
- Hexadecimal to decimal conversion:
 - Convert $0x4AF$ to decimal

Hexadecimal to Binary Conversion

- Hexadecimal to binary conversion:
 - Convert $4AF_{16}$ (also written $0x4AF$) to binary
 - $0100\ 1010\ 1111_2$
- Hexadecimal to decimal conversion:
 - Convert $4AF_{16}$ to decimal
 - $16^2 \times 4 + 16^1 \times 10 + 16^0 \times 15 = 1199_{10}$

Conversion between numbers of different representations

- Convert $(10D)_{16}$ to their decimal format

$$\begin{aligned}(10D)_{16} &= 1 \times 16^2 + 13 \times 16^0 \\ &= 256 + 13 = 269\end{aligned}$$

Conversion between numbers of different representations

1) Conversion from binary (octal, hexadecimal) to decimal

(按权展开法：先写成多项式，然后计算十进制结果)

$$\begin{aligned} N &= d_{n-1}d_{n-2}\cdots d_1d_0.d_{-1}d_{-2}\cdots d_{-m} \\ &= d_{n-1} \times R^{n-1} + d_{n-2} \times R^{n-2} + \cdots + d_1 \times R^1 + d_0 \times R^0 \quad (\text{integer part}) \\ &\quad + d_{-1} \times R^{-1} + d_{-2} \times R^{-2} + \cdots + d_{-m} \times R^{-m} \quad (\text{decimal part}) \end{aligned}$$

where r denotes the base

- $R=2$;
- $R=8$;
- $R=16$;

Conversion between numbers of different representations

2) decimal to binary

- Two steps:
 - Integer part (除2取余法)
 - Based on repeated decimal division by 2
 - Fractional part (乘2取整法)
 - Based on repeated decimal multiplication by 2

Conversion between numbers of different representations

2) decimal to binary

example

Integer part

Convert $(327)_{10}$ to binary

2		327		
2		163	1	low
2		81	1	
2		40	1	
2		20	0	
2		10	0	
2		5	0	
2		2	1	
2		1	0	
2		0	1	high
$(327)_{10} = (101000111)_2$				

Conversion between numbers of different representations

2) decimal to binary

example

Fractional part

Convert $(0.8125)_{10}$ to binary

$0.8125 \times 2 = 1.625$	1	high
$0.625 \times 2 = 1.25$	1	
$0.25 \times 2 = 0.5$	0	
$0.5 \times 2 = 1$	1	low

$$(0.8125)_{10} = (0.1101)_2$$

Conversion between numbers of different representations

2) decimal to binary

Convert $(0.2)_{10}$ to binary

example

Fractional part

$$0.2 \times 2 = 0.4 \quad \mathbf{0} \quad \text{.....high}$$

$$0.4 \times 2 = 0.8 \quad \mathbf{0}$$

$$0.8 \times 2 = 1.6 \quad \mathbf{1}$$

$$0.6 \times 2 = 1.2 \quad \mathbf{1}$$

$$0.2 \times 2 = 0.4 \quad \mathbf{0}$$

$$0.4 \times 2 = 0.8 \quad \mathbf{0}$$

$$0.8 \times 2 = 1.6 \quad \mathbf{1}$$

$$0.6 \times 2 = 1.2 \quad \mathbf{1} \quad \text{.....low}$$

$$(0.2)_{10} = [0.001100110011....]_2$$

Conversion between numbers of different representations

3) others

binary ↔ octal			binary ↔ hexadecimal			
000	0		0000	0	1000	8
001	1		0001	1	1001	9
010	2		0010	2	1010	A
011	3		0011	3	1011	B
100	4		0100	4	1100	C
101	5		0101	5	1101	D
110	6		0110	6	1110	E
111	7		0111	7	1111	F

Conversion between numbers of different representations

Ex

convert binary to octal

$$(10110111.01101)_2 = (267.32)_8$$

binary: 10,110,111.011,01

binary: 010,110,111.011,010

octal: 2 6 7 . 3 2

Ex

Ex 2 : convert octal to binary

$$(123.46)_8$$

$$= (001,010,011.100,110)_2$$

$$= (1010011.10011)_2$$

Conversion between numbers of different representations

Ex

convert binary to hexadecimal

$$(110110111.01101)_2 = (1B7.68)_{16}$$

binary: 1,1011,0111.0110,1

binary: 0001,1011,0111.0110,1000

Hexa-: 1 B 7 . 6 8

Ex

Ex 4: convert hexadecimal to binary

$$(7AC.DE)_{16}$$

$$= (0111,1010,1100.1101,1110)_2$$

$$= (11110101100.1101111)_2$$

Bits, Bytes, Nibbles...

- Bits

10010110
[] []
most least
significant significant
bit bit
(msb) (lsb)

- Byte & Nibbles

byte
10010110
[]
nibble

- word (Bytes)

CEBF9AD7
[] []
most least
significant significant
Byte Byte
(MSB) (LSB)

8 位 经典 游戏 卡







Large Powers of Two

- $2^{10} = 1 \text{ kilo} \approx 1000 \text{ (1024)}$
- $2^{20} = 1 \text{ mega} \approx 1 \text{ million (1,048,576)}$
- $2^{30} = 1 \text{ giga} \approx 1 \text{ billion (1,073,741,824)}$

Estimating Powers of Two

- What is the value of 2^{24} ?
- How many values can a 32-bit variable represent?

Estimating Powers of Two

- What is the value of 2^{24} ?
 - $2^4 \times 2^{20} \approx 16$ million
- How many values can a 32-bit variable represent?
 - $2^2 \times 2^{30} \approx 4$ billion

Addition

- Decimal

$$\begin{array}{r} 11 \leftarrow \text{carries} \\ 3734 \\ + 5168 \\ \hline 8902 \end{array}$$

- Binary

$$\begin{array}{r} 11 \leftarrow \text{carries} \\ 1011 \\ + 0011 \\ \hline 1110 \end{array}$$

Binary Addition Examples

- Add the following 4-bit binary numbers

$$\begin{array}{r} 1001 \\ + 0101 \\ \hline \end{array}$$

- Add the following 4-bit binary numbers

$$\begin{array}{r} 1011 \\ + 0110 \\ \hline \end{array}$$

Binary Addition Examples

- Add the following 4-bit binary numbers

$$\begin{array}{r} 1 \\ 1001 \\ + 0101 \\ \hline 1110 \end{array}$$

- Add the following 4-bit binary numbers

$$\begin{array}{r} 111 \\ 1011 \\ + 0110 \\ \hline 10001 \end{array}$$

Overflow!

Overflow

- Digital systems operate on a **fixed number of bits**
- Overflow: when result is too big to fit in the available number of bits
- See previous example of $11 + 6$

Overflow judgment

which situation causes overflow?

- Overflow: result exceeds the range of representation.
- Obviously, addition with two opposite sign numbers and subtraction with two same sign numbers will not cause overflow.
- Overflow must be caught.

Overflow judgment

Assume two signed numbers

$$[A]_{2's\ compl} = [S_A A_{n-2} \dots A_1 A_0]_{2's\ compl}$$

$$[B]_{2's\ compl} = [S_B B_{n-2} \dots B_1 B_0]_{2's\ compl}$$

$$\begin{array}{rcccc}
 S_A & A_3 & A_2 & A_1 & A_0 \\
 C_F S_B C & B_3 & B_2 & B_1 & B_0 \\
 \hline
 S_F & F_3 & F_2 & F_1 & F_0
 \end{array}$$

■ Overflow judgment

(1) A=3 B=2

$$\begin{array}{r} 3+2: \quad 0 \ 0011 \\ \quad \quad 0 \ 0010 \quad + \\ \hline \quad \quad 0 \ 0101 \quad \text{correct} \end{array}$$

(2) A=10 B=7

$$\begin{array}{r} 10+7: \quad 0 \ 1010 \\ \quad \quad 0 \ 0111 \quad + \\ \hline \quad \quad 1 \ 0001 \quad \text{overflow} \end{array}$$

(3) A= -3 B= -2

$$\begin{array}{r} -3+(-2): \quad 1 \ 1101 \\ \quad \quad 1 \ 1110 \quad + \\ \hline \quad \quad 1 \ 1011 \quad \text{correct} \end{array}$$

(4) A= -10 B= -7

$$\begin{array}{r} -10+(-7): \quad 1 \ 0110 \\ \quad \quad 1 \ 1001 \quad + \\ \hline \quad \quad 0 \ 1111 \quad \text{overflow} \end{array}$$

(5) A=6 B= -4

$$\begin{array}{r} 6+(-4): \quad 0 \ 0110 \\ \quad \quad 1 \ 1100 \quad + \\ \hline \quad \quad 0 \ 0010 \quad \text{correct} \end{array}$$

(6) A= -6 B=4

$$\begin{array}{r} -6+4: \quad 1 \ 1010 \\ \quad \quad 0 \ 0100 \quad + \\ \hline \quad \quad 1 \ 1110 \quad \text{correct} \end{array}$$

■ Overflow judgment

① Judgment logic by hardware 1 (SA、SB and Sf)

SA	SB	Sf	Overflow
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

$$OF = \overline{S_A} \overline{S_B} S_f + S_A S_B \overline{S_f}$$

② Judgment logic by hardware 2 (Cf and C)

■ Overflow judgment

(1) A=3 B=2

$$\begin{array}{r} 3+2: \quad 0 \ 0011 \\ \quad C_f=0 \quad 0 \ 0010 \quad + \\ \hline \quad C=0 \quad 0 \ 0101 \quad \text{correct} \end{array}$$

(2) A=10 B=7

$$\begin{array}{r} 10+7: \quad 0 \ 1010 \\ \quad C_f=0 \quad 0 \ 1011 \quad + \\ \hline \quad C=1 \quad 1 \ 0001 \quad \text{overflow} \end{array}$$

(3) A=-3 B=-2

$$\begin{array}{r} -3+(-2): \quad 1 \ 1101 \\ \quad C_f=1 \quad 1 \ 11110 \quad + \\ \hline \quad C=1 \quad 1 \ 1011 \quad \text{correct} \end{array}$$

(4) A=-10 B=-7

$$\begin{array}{r} -10+(-7): \quad 1 \ 0110 \\ \quad C_f=1 \quad 1 \ 11001 \quad + \\ \hline \quad C=0 \quad 0 \ 1111 \quad \text{overflow} \end{array}$$

(5) A=6 B=-4

$$\begin{array}{r} 6+(-4): \quad 0 \ 0110 \\ \quad C_f=1 \quad 1 \ 11100 \quad + \\ \hline \quad C=1 \quad 0 \ 0010 \quad \text{correct} \end{array}$$

(6) A=-6 B=4

$$\begin{array}{r} -6+4: \quad 1 \ 1010 \\ \quad C_f=0 \quad 0 \ 0100 \quad + \\ \hline \quad C=0 \quad 1 \ 1110 \quad \text{correct} \end{array}$$

■ Overflow judgment

① Judgment logic by hardware 1 (SA、SB and Sf)

$$OF = \bar{S}_A \bar{S}_B S_f + S_A S_B \bar{S}_f$$

② Judgment logic by hardware 2 (Cf and C)

$$OF = C_f \oplus C$$

③ Judgment logic by hardware 3 (double sign bit)

■ Overflow judgment

(1) 3+2:

$$\begin{array}{r} 00\ 0011 \\ 00\ 0010 \\ \hline 00\ 0101 \end{array} \quad \begin{array}{l} + \\ \\ \text{correct} \end{array}$$

(2) 10+7:

First Sfl

$$\begin{array}{r} 00\ 1010 \\ 00\ 0111 \\ \hline 01\ 0001 \end{array} \quad \begin{array}{l} + \\ \\ \text{overflow} \end{array}$$

(3) -3+(-2):

$$\begin{array}{r} 11\ 1101 \\ 11\ 1110 \\ \hline 11\ 0111 \end{array} \quad \begin{array}{l} + \\ \\ \text{correct} \end{array}$$

(4) -10+(-7):

Second Sf2

$$\begin{array}{r} 11\ 0110 \\ 11\ 1001 \\ \hline 10\ 1111 \end{array} \quad \begin{array}{l} + \\ \\ \text{underflow} \end{array}$$

(5) 6+(-4):

$$\begin{array}{r} 00\ 0110 \\ 11\ 1100 \\ \hline 00\ 0010 \end{array} \quad \begin{array}{l} + \\ \\ \text{correct} \end{array}$$

(6) -6+4:

$$\begin{array}{r} 11\ 1010 \\ 00\ 0100 \\ \hline 11\ 1110 \end{array} \quad \begin{array}{l} + \\ \\ \text{correct} \end{array}$$

■ Overflow judgment

① Judgment logic by hardware 1 (S_A , S_B and S_f)

$$OF = \bar{S}_A \bar{S}_B S_f + S_A S_B \bar{S}_f$$

② Judgment logic by hardware 2 (C_f and C)

$$OF = C_f \oplus C$$

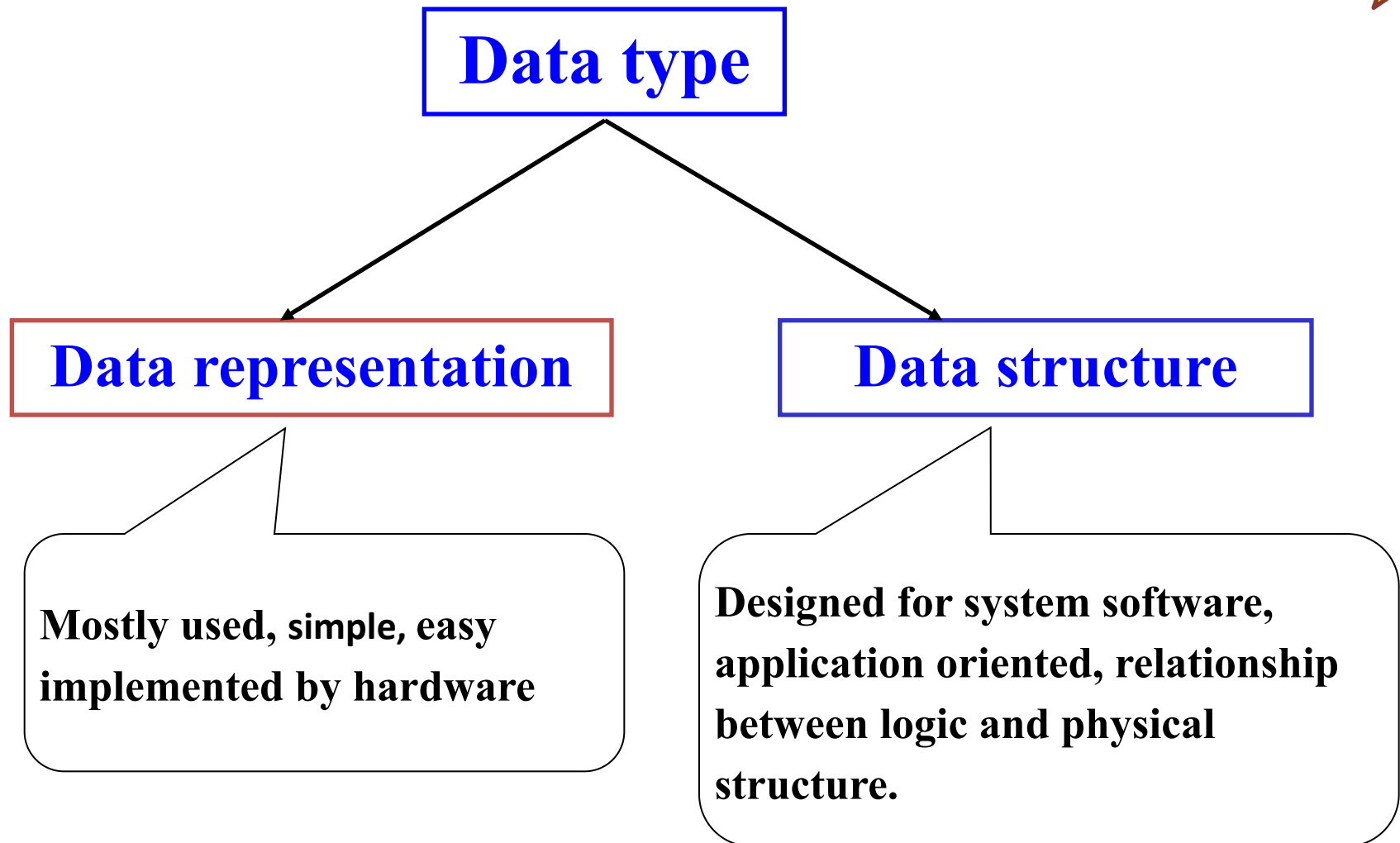
③ Judgment logic by hardware 3 (double sign bit)

$$OF = S_{f1} \oplus S_{f2}$$

The representation of information in a computer

- **Data types** in a computer system:
 - file, graph, table, tree, array, queue, linked list, stack, vector, string, real, integer, Boolean, char
- This chapter makes no attempt to discuss the representations of all kinds of information. It provides a basic introduction to representing information.

The representation of information in a computer



The representation of information in a computer



- Principles to decide which data types are implemented by data representation:
 1. Reduce the running time of program
 2. Reduce the communication cost between CPU and memory
 3. Universality and Utilization (??? IEEE standard)

Data types representing information in a computer

- Binary variables and binary codes serve as the basis for representing information
 - In order to represent different types of information, we need to apply different methods and systems of representation utilizing binary digits.
 - Unsigned numbers
 - Signed numbers
 - Bit string
- Fix-Point Number**
Floating-Point Number

Data types representing information in a computer

- Unsigned number: are created directly as ordinary binary codes
 - Operation codes in the OP code field (?)
 - Register numbers in address field (?)
 - Memory address (?)
 - counter
 - Time, clock frequency

Data types representing information in a computer

- Signed number
 - Numerical and engineering computation
 - Most arithmetic operations in the instruction set handle signed integers or floating-point numbers as their operands.
- bit string (has an even wider application)
 - logic operations in the instruction set
 - Denote symbols such as characters
 - In control unit of a computer, for example, in status words, tagged data, and control codes.

Data types representing information in a computer

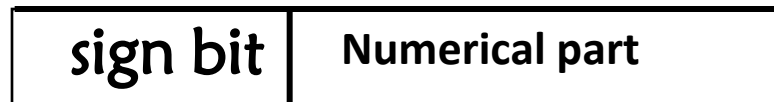
- Fix-Point Number

- Point means the Decimal point in binary. Fix-point means the position of Decimal point is fixed.
- It is hidden (不占位)
- consists of fix-point decimal and fix-point integer

F-p integer



F-p decimal



Position of
dicemal point

Signed Binary Numbers

- How to denote the sign of a number
 - ‘0’ for positive number
 - ‘1’ for negative number
 - A separate sign bit is placed at **msb** position
- Sign-Magnitude Numbers (原码)
- Two’s Complement Numbers (补码)

Sign/Magnitude Numbers

- 1 sign bit, $N-1$ magnitude bits
- Sign bit is the most significant (left-most) bit
 - Positive number: sign bit = 0 $A : \{a_{N-1}, a_{N-2}, \dots, a_2, a_1, a_0\}$
 - Negative number: sign bit = 1 $A = (-1)^{a_{n-1}} \sum_{i=0}^{n-2} a_i 2^i$
- Example, 4-bit sign/mag representations of ± 6 :
 - +6 =
 - 6 =
- Range of an N -bit sign/magnitude number:

Sign/Magnitude Numbers

- 1 sign bit, $N-1$ magnitude bits
 - Sign bit is the most significant (left-most) bit
 - Positive number: sign bit = 0 $A : \{a_{N-1}, a_{N-2}, \dots, a_2, a_1, a_0\}$
 - Negative number: sign bit = 1
- $$A = (-1)^{a_{n-1}} \sum_{i=0}^{n-2} a_i 2^i$$
- Example, 4-bit sign/mag representations of ± 6 :
 - +6 = **0110**
 - 6 = **1110**
 - Range of an N -bit sign/magnitude number:
 $[-(2^{N-1}-1), 2^{N-1}-1]$

Sign-magnitude representation (原码)

- decimal

Example:

$$X_1 = + 0.1011011$$

$$[X_1]_{\text{sign mag}} = \mathbf{0.1011011}$$

$$X_2 = - 0.1011011$$

$$[X_2]_{\text{sign mag}} = \mathbf{1.1011011}$$

Sign-magnitude representation (原码)

- Integer

Example:

$$X_1 = + 0\ 1011011$$

$$[X_1]_{\text{sign-mag}} = \mathbf{01011011}$$

$$X_2 = - 0\ 1011011$$

$$[X_2]_{\text{sign-mag}} = \mathbf{11011011}$$

Sign-magnitude representation (原码)

- Range of decimal (n 位机器字长)

- $[+0] = 0.00000000$; $[-0] = 1.00000000$
- $-(1-2^{-(n-1)}) \sim 1-2^{-(n-1)}$
- Number : $2^n - 1$

- Length of word is 8:

- range: $-(1-2^{-7}) \sim (1-2^{-7})$
- $-127/128 \sim 127/128$
- amount: 255

Sign-magnitude representation (原码)

- Range of decimal (n 位机器字长)

- $[+0] = 0.00000000$; $[-0] = 1.00000000$
- $-(1-2^{-(n-1)}) \sim 1-2^{-(n-1)}$
- Number : $2^n - 1$

- Length of word is 8:

- range: $-(1-2^{-7}) \sim (1-2^{-7})$
- $-127/128 \sim 127/128$,
- amount: 255

1.1111111

0.1111111

Sign-magnitude representation (原码)

- Range of integer

- $[+0] = 00000000$; $[-0] = 10000000$
- range : $-(2^{n-1}-1) \sim 2^{n-1}-1$
- Numbers : $2^n - 1$

- Word length 8: max 127, min -127, number 255
- Word length 16: max 32767, min -32767, number 65535



Sign/Magnitude Numbers

- Problems:
 - Addition doesn't work, for example $-6 + 6$:

$$\begin{array}{r} 1110 \\ + 0110 \\ \hline 10100 \text{ (wrong!)} \end{array}$$

- Two representations of 0 (± 0):

1000

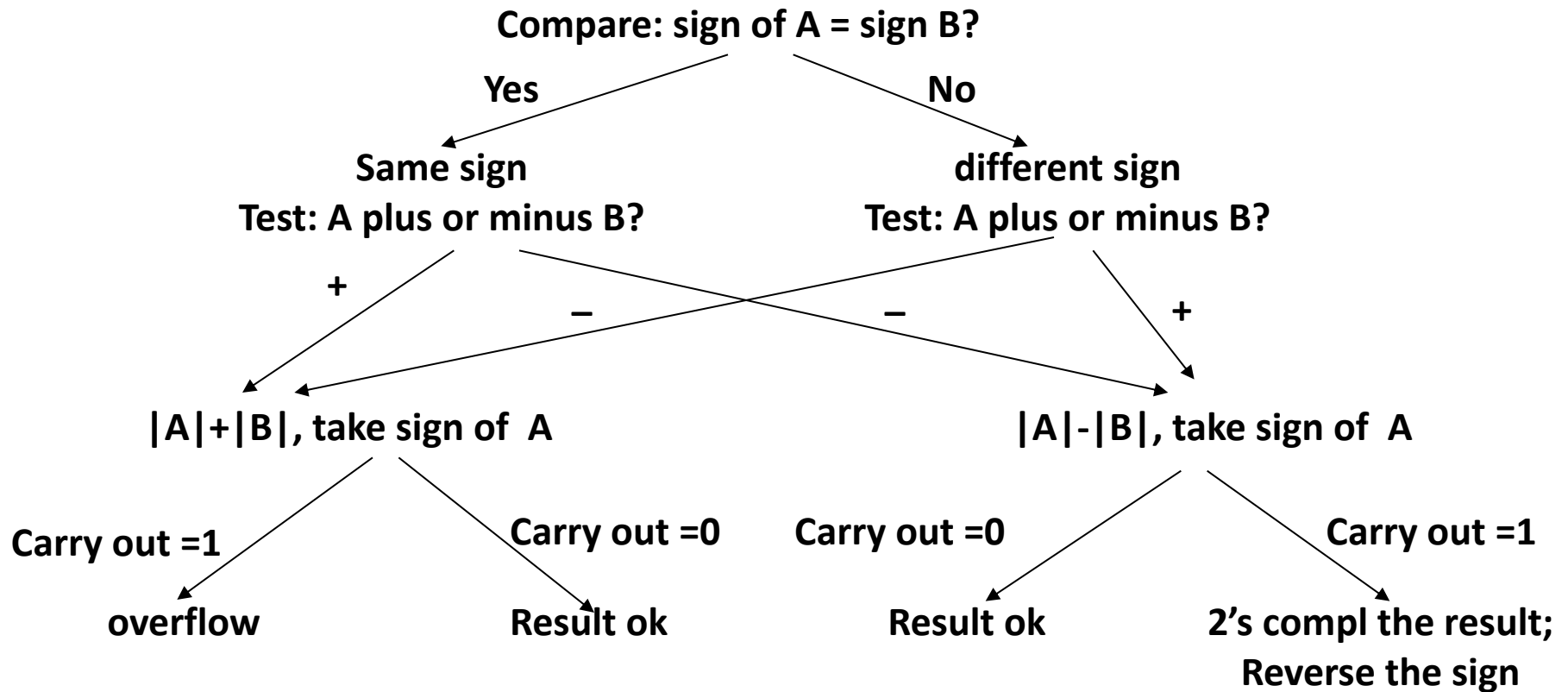
0000

Sign-magnitude representation (原码)

- Character of sign-magnitude representation
 - Simple to represent, easy to multiply/divide operation
 - Both +0 and -0 exist, it can be troublesome to have two different representations for the same number.
 - Hard to add/sub operation
 - Requires comparing the sign bits of two operands
 - An alternative way, 2's complement representation(补码)

Sign-magnitude representation (原码)

- Sign-magnitude add/sub



the flowchart of the algorithm for sign-magnitude add/sub

Two's Complement Numbers(补码)

- Don't have same problems as sign/magnitude numbers:
 - Addition works
 - Single representation for 0

Two's Complement Numbers

- Msb has value of -2^{N-1}

$$A = a_{n-1} \left(-2^{n-1} \right) + \sum_{i=0}^{n-2} a_i 2^i$$

- Biggest positive 4-bit number:
- Smallest negative 4-bit number:
- The most significant bit still indicates the sign (1 = negative, 0 = positive)
- Range of an N -bit two's comp number:

Two's Complement Numbers

- Msb has value of -2^{N-1}

$$A = a_{n-1} \left(-2^{n-1} \right) + \sum_{i=0}^{n-2} a_i 2^i$$

- Biggest positive 4-bit number: **0111**
- Smallest negative 4-bit number: **1000**
- The most significant bit still indicates the sign (1 = negative, 0 = positive)
- Range of an N -bit two's comp number:

$$[-(2^{N-1}), 2^{N-1}-1]$$

“Taking the Two’s Complement”

- Flip the sign of a two’s complement number
- Method:
 1. Invert the bits
 2. Add 1
- Example: Flip the sign of $3_{10} = 0011_2$

“Taking the Two’s Complement”

- Flip the sign of a two’s complement number
- Method:
 1. Invert the bits
 2. Add 1
- Example: Flip the sign of $3_{10} = 0011_2$

1. 1100

2. $\begin{array}{r} + \quad 1 \\ \hline \end{array}$

$1101 = -3_{10}$

Two's complement representation(补码)

- origin:

transform the subtract operation to add:

$$\begin{array}{r} 253 \\ - 176 \\ \hline ??? \end{array}$$

$$\begin{array}{r} 999 \\ - 176 \\ \hline 823 \end{array}$$

Two's complement representation(补码)

$$\begin{array}{r} 253 \\ - 176 \\ \hline 77 \end{array}$$

$$\begin{array}{r} + 253 \\ - 999+1 \\ - 176 \\ - 1000 \\ \hline \end{array}$$

Sub
To
add

$$\begin{array}{r} + 253 \\ + 823+1 \\ \hline 1\ 077 \end{array}$$

824 is complement of -176 with respect to 1000 (!!!???)

Two's complement representation(补码)

- $253 - 176 = 77$

$$253_{10} = 11111101_2; \quad 176_{10} = 10110000_2; \quad 77_{10} = 01001101_2;$$

$$\begin{array}{r} 11111101 \\ - 10110000 \\ \hline 01001101 \end{array}$$

$$\begin{array}{r} 11111101 \\ + 11111111 + 1 \\ - 10110000 \\ \hline \end{array}$$

$$\begin{array}{r} 11111101 \\ + 01001111 + 1 \\ \hline 1\ 01001100 + 1 \\ \hline = 01001101 \end{array}$$

↙
Add the complement,

If the result is overflow, the highest position is dropped automatically.

Two's complement representation(补码)

- 补码的补充说明：

就象我们前面所演示给大家看的，补码的发现其实是为了消灭减法。正数根本不需要什么补码，补码是一个减法的差，所以求一个数的补码就是做一个减法。

Two's complement representation(补码)

- Decimal

Example:

$$X_1 = + 0.1011011$$

$$[X_1]_{2's\ compl} = 0.1011011$$

$$X_2 = - 0.1011011$$

$$[X_2]_{2's\ compl} = 1.0100101$$

Two's complement representation(补码)

- Integer

Example:

$$X_1 = + 01011011$$

$$[X_1]_{2's\ compl} = 01011011$$

$$X_2 = - 01011011$$

$$[X_2]_{2's\ compl} = 10100101$$

Two's Complement Examples

- Take the two's complement of $6_{10} = 0110_2$
- What is the decimal value of 1001_2 ?

Two's Complement Examples

- Take the two's complement of -6 ($6_{10} = 0110_2$)

1. 1001

2. $\begin{array}{r} + \quad 1 \\ \hline \end{array}$

$1010_2 = -6_{10}$

- What is the decimal value of the two's complement number 1001_2 ?

1. 0110

2. $\begin{array}{r} + \quad 1 \\ \hline \end{array}$

$0111_2 = 7_{10}$, so $1001_2 = -7_{10}$

Two's Complement Examples

- Take the two's complement of $0_{10} = 0000_2$

Two's Complement Examples

- Take the two's complement of $0_{10} = 0000_2$

1. 1111

2. + 1

$$\underline{\hspace{1cm}} \\ 1\ 0000_2 = -0_{10}$$

- Hence, zero is always represented with all 0's. Unlike the sign-magnitude system.
- The two's complement system has no separate -0. Zero is considered positive because its sign bit is 0.

Two's Complement Examples

- there is one more negative number than positive number because there is no -0.
- **The most negative number** $10...000_2 = -2^{N-1}$ is sometimes called the weird number.(怪异数)
- Its two's complement is found by inverting the bits (producing $01...111_2$ and adding 1, which produces $10...000_2$, the weird number, again).
- Hence, this negative number has no positive counterpart.

Two's complement representation(补码)

- Range of complement representation:

- integer: -2^{n-1} to $2^{n-1} - 1$

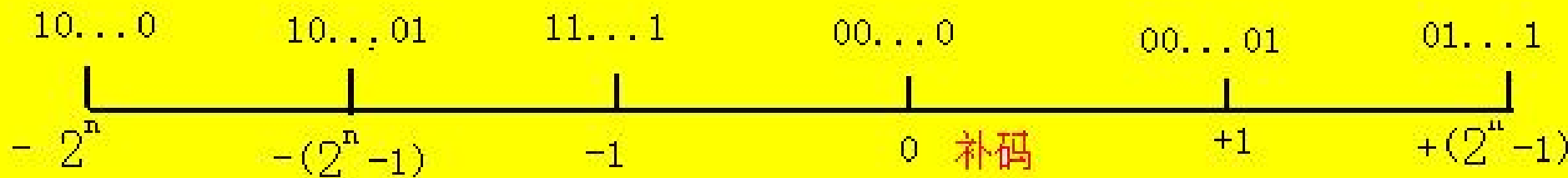
- decimal: - 1 to $1 - 2^{-n-1}$

- Number: 2^n

Word length $n=8$:

Range: $-128 \sim 127$

$-1 \sim 127/128$



Two's Complement Addition

- Add $6 + (-6)$ using two's complement numbers

$$\begin{array}{r} 0110 \\ + 1010 \\ \hline \end{array}$$

- Add $-2 + 3$ using two's complement numbers

$$\begin{array}{r} 1110 \\ + 0011 \\ \hline \end{array}$$

Two's Complement Addition

- Add $6 + (-6)$ using two's complement numbers

$$\begin{array}{r} 111 \\ 0110 \\ + 1010 \\ \hline 10000 \end{array}$$

- Add $-2 + 3$ using two's complement numbers

$$\begin{array}{r} 111 \\ 1110 \\ + 0011 \\ \hline 10001 \end{array}$$

Two's Complement Addition

- Transform sub to add
 - $6-5$
 $= 6+(-5)$

Increasing Bit Width

- **Extend number from N to M bits ($M > N$) :**
 - Sign-extension
 - Zero-extension

Sign-Extension

- Sign bit copied to msb's
- Number value is same

- **Example 1:**

- 4-bit representation of 3 = 0011
- 8-bit sign-extended value: 00000011

- **Example 2:**

- 4-bit representation of -5 = 1011
- 8-bit sign-extended value: 11111011

Zero-Extension

- Zeros copied to msb's
- Value changes for negative numbers

- **Example 1:**

- 4-bit value = $0011_2 = 3_{10}$
- 8-bit zero-extended value: $00000011 = 3_{10}$

- **Example 2:**

- 4-bit value = $1011 = -5_{10}$
- 8-bit zero-extended value: $00001011 = 11_{10}$

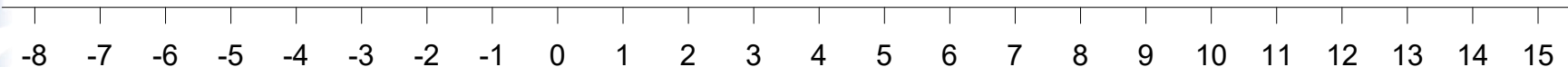
Bit-Extension

- Why we need extension
 - precision change
 - type change

Number System Comparison

Number System	Range
Unsigned	$[0, 2^N-1]$
Sign/Magnitude	$[-(2^{N-1}-1), 2^{N-1}-1]$
Two's Complement	$[-2^{N-1}, 2^{N-1}-1]$

For example, 4-bit representation:



Unsigned

0000 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100 1101 1110 1111

1000 1001 1010 1011 1100 1101 1110 1111 0000 0001 0010 0011 0100 0101 0110 0111

Two's Complement

1111 1110 1101 1100 1011 1010 1001 0000
1000 0001 0010 0011 0100 0101 0110 0111

Sign/Magnitude

One's complement representation(反码)

- positive the sign '0' at msb is attached to the magnitude in the remaining n-1 bits.
- Negative the sign '1' at msb is attached to the 1's compl of the magnitude in the remaining n-1 bits.

example

- decimal:

$$-X_1 = +0.1011011, [X_1]_{1's\ compl} = 0.1011011$$

$$-X_2 = -0.1011011, [X_2]_{1's\ compl} = 1.0100100$$

- integer:

$$-X_3 = +1011011, [X_3]_{1's\ compl} = 01011011$$

$$-X_4 = -1011011, [X_4]_{1's\ compl} = 10100100$$

- $[+0]_{1's\ compl} = 00000000$; $[-0]_{1's\ compl} = 11111111$

Give the sign-mag 、 1's compl 、 2's compl representation of the follow number (word length is 8), where msb is the highest bit (sign bit), lsb is the lowest bit. If the number is decimal, the decimal point is behind the msb; if the number is integer, then the decimal point is behind the lsb.

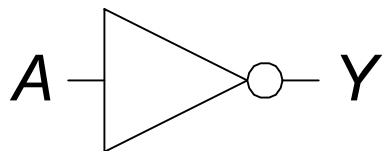
- (1) $-35/64$
- (2) $23/128$
- (3) -127
- (4) -1 (decimal representation)
- (5) -1 (integer representation)

Chapter 1 :: Topics

- Background
- The Game Plan
- The Art of Managing Complexity
- The Digital Abstraction
- Number Systems
- **Logic Gates**
- Logic Levels
- CMOS Transistors
- Power Consumption

Single-Input Logic Gates

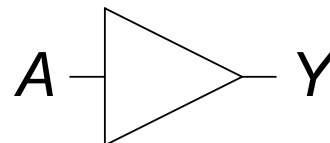
NOT



$$Y = \overline{A}$$

A	Y
0	
1	

BUF



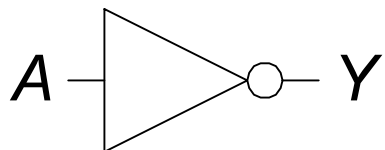
$$Y = A$$

A	Y
0	
1	

Single-Input Logic Gates



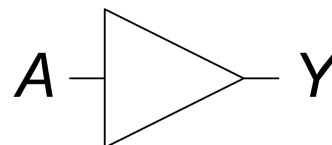
NOT



$$Y = \overline{A}$$

A	Y
0	1
1	0

BUF



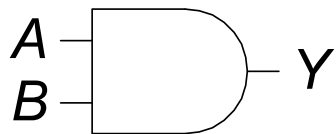
?!

$$Y = A$$

A	Y
0	0
1	1

Two-Input Logic Gates

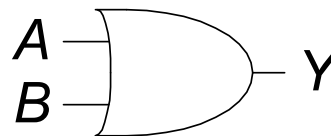
AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR

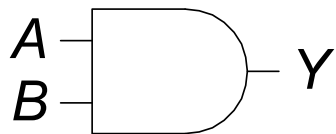


$$Y = A + B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Two-Input Logic Gates

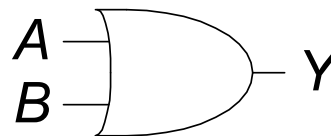
AND



$$Y = AB$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR



$$Y = A + B$$

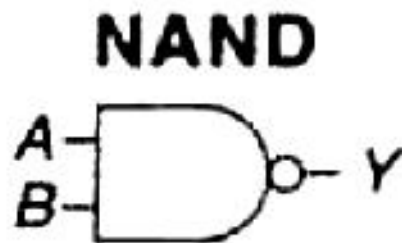
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Two-Input Logic Gates



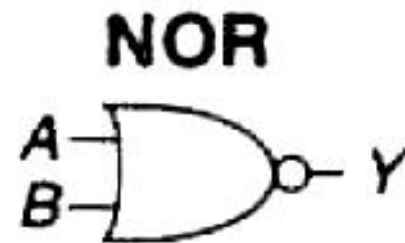
$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

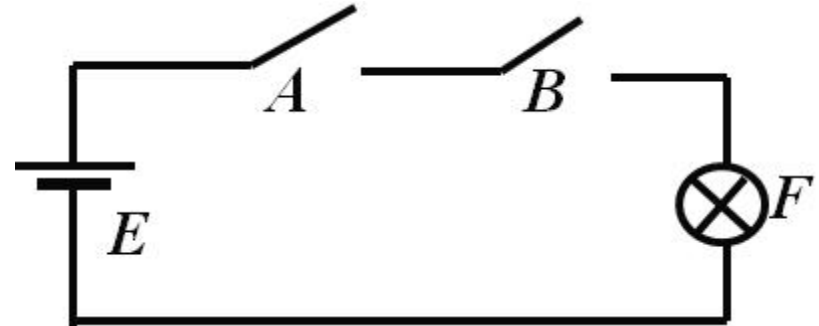
其他两输入逻辑门

- Boolean algebra and logic expression
 - Algebra of binary variables and functions, named after a 19th-century mathematician George Boole.
 - Logic expression is an algebraic expression composed of literals of **independent variables** and **logic operators** acting on these literals.
 - All logic functions can be represented as the functional compositions of these three logic operations (AND,OR,NOT)
 - Using the **truth table** is the simplest and accurate method of defining a logic function.

- Boolean algebra and logic expression

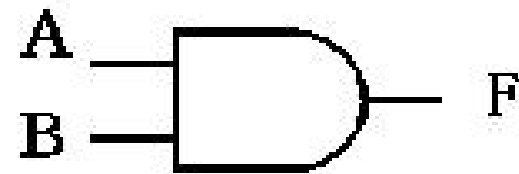
- **AND Logic**

$$F = A \cdot B = A \wedge B = AB$$

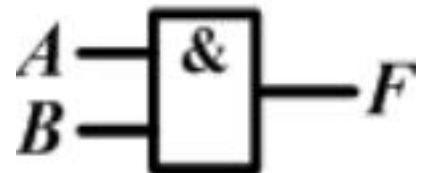
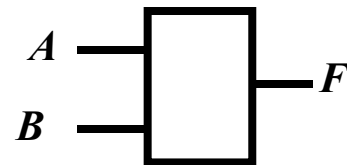


truth table

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1



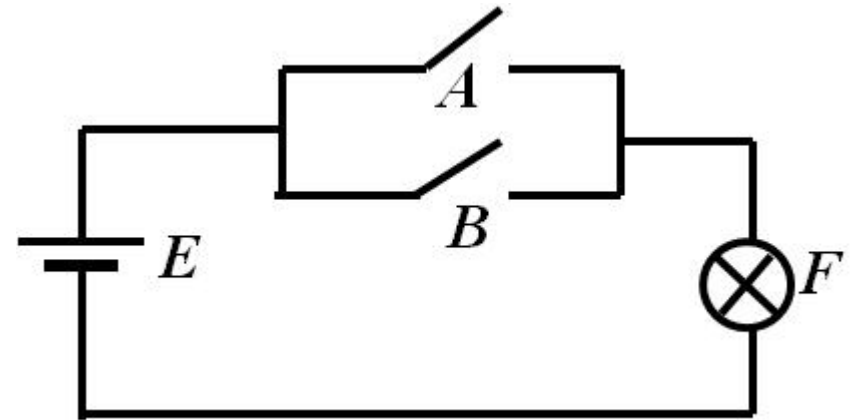
(a) 与门



- Boolean algebra and logic expression

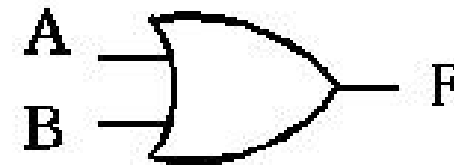
- OR Logic

$$F = A + B = A \vee B = A/B$$

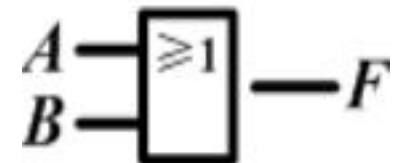
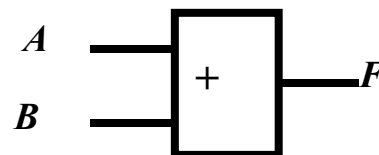


truth table

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1



(b) 或门

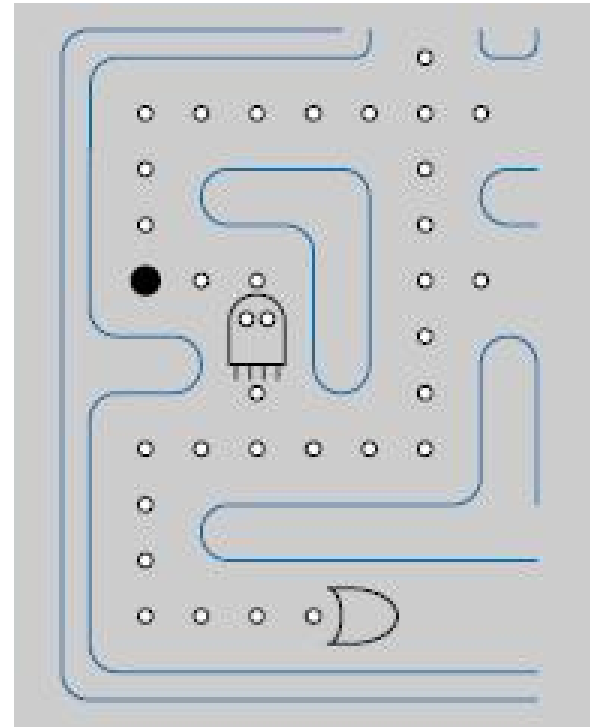
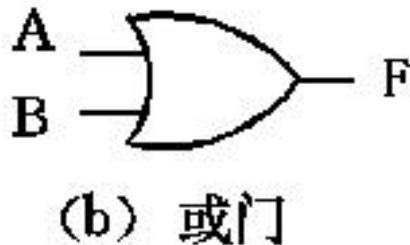


- Boolean algebra and logic expression

- **OR Logic**

$$F = A + B = A \vee B = A/B$$

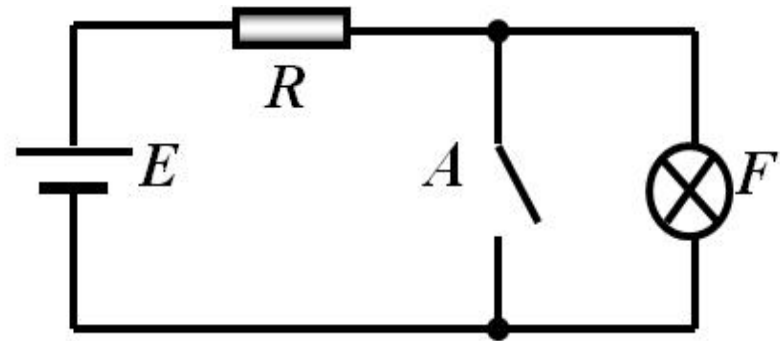
- A silly way to remember the OR symbol is that its input side is curved like Pacman's mouth, so the gate is hungry and willing to eat any TRUE inputs it can find!



- Boolean algebra and logic expression

- NOT Logic

$$F = \bar{A} = A' = !A = \sim A = \neg A$$



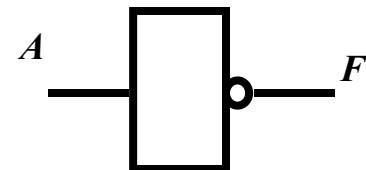
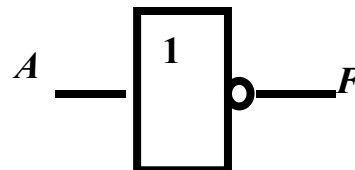
truth table

A	F
0	1
1	0



(c) 反相门

inverter

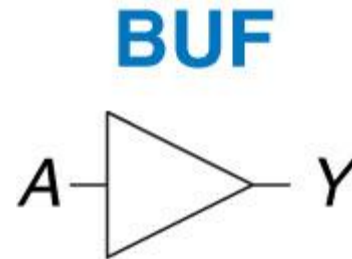


- Boolean algebra and logic expression
 - **Buffer**

$$F=A$$

truth table

<i>A</i>	<i>F</i>
0	0
1	1

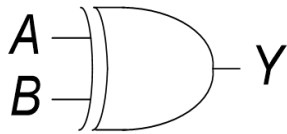


Logic Gates

- **Perform logic functions:**
 - inversion (NOT), AND, OR, NAND, NOR, etc.
- **Single-input:**
 - NOT gate, buffer
- **Two-input:**
 - AND, OR, XOR, NAND, NOR, XNOR
- **Multiple-input**

More Two-Input Logic Gates

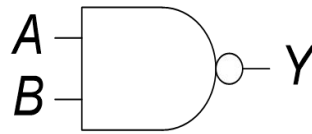
XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

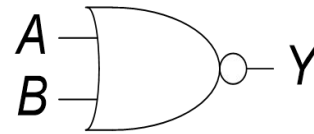
NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

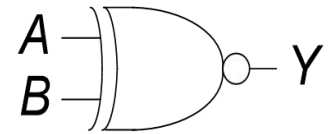
NOR



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR

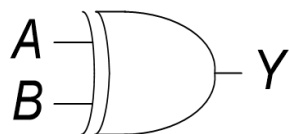


$$Y = \overline{A \oplus B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

More Two-Input Logic Gates

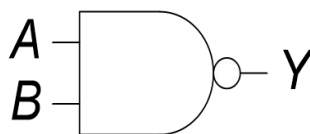
XOR



$$Y = A \oplus B$$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

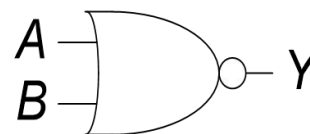
NAND



$$Y = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

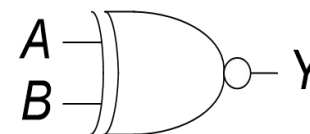
NOR



$$Y = \overline{A + B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

XNOR

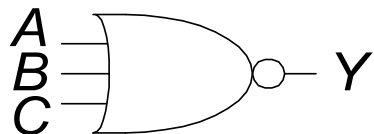


$$Y = \overline{A \oplus B}$$

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Multiple-Input Logic Gates

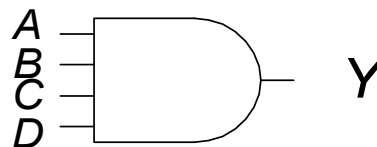
NOR3



$$Y = \overline{A+B+C}$$

A	B	C	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

AND4

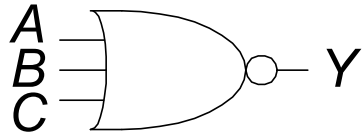


$$Y = ABCD$$

A	B	C	Y
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

Multiple-Input Logic Gates

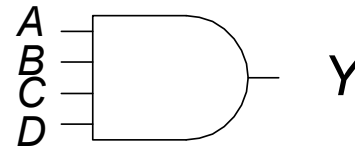
NOR3



$$Y = \overline{A+B+C}$$

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

AND4



$$Y = ABCD$$

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

- Multi-input XOR: Odd parity

Chapter 1 :: Topics

- Background
- The Game Plan
- The Art of Managing Complexity
- The Digital Abstraction
- Number Systems
- Logic Gates
- **Logic Levels**
- CMOS Transistors
- Power Consumption

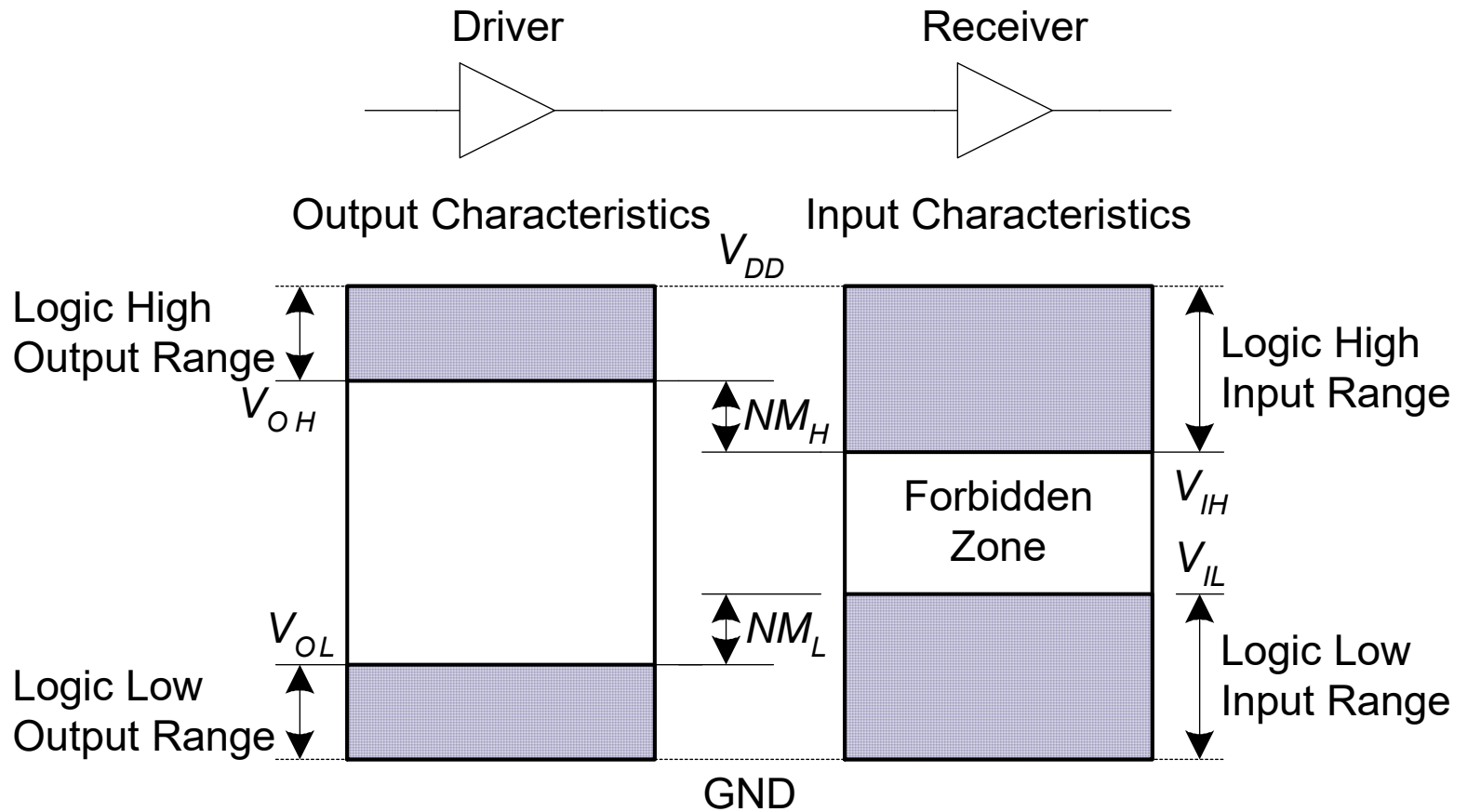
Logic Levels

- Discrete voltages represent 1 and 0
- For example:
 - 0 = *ground* (GND) or 0 volts
 - 1 = V_{DD} or 5 volts
- What about 4.99 volts? Is that a 0 or a 1?
- What about 3.2 volts?

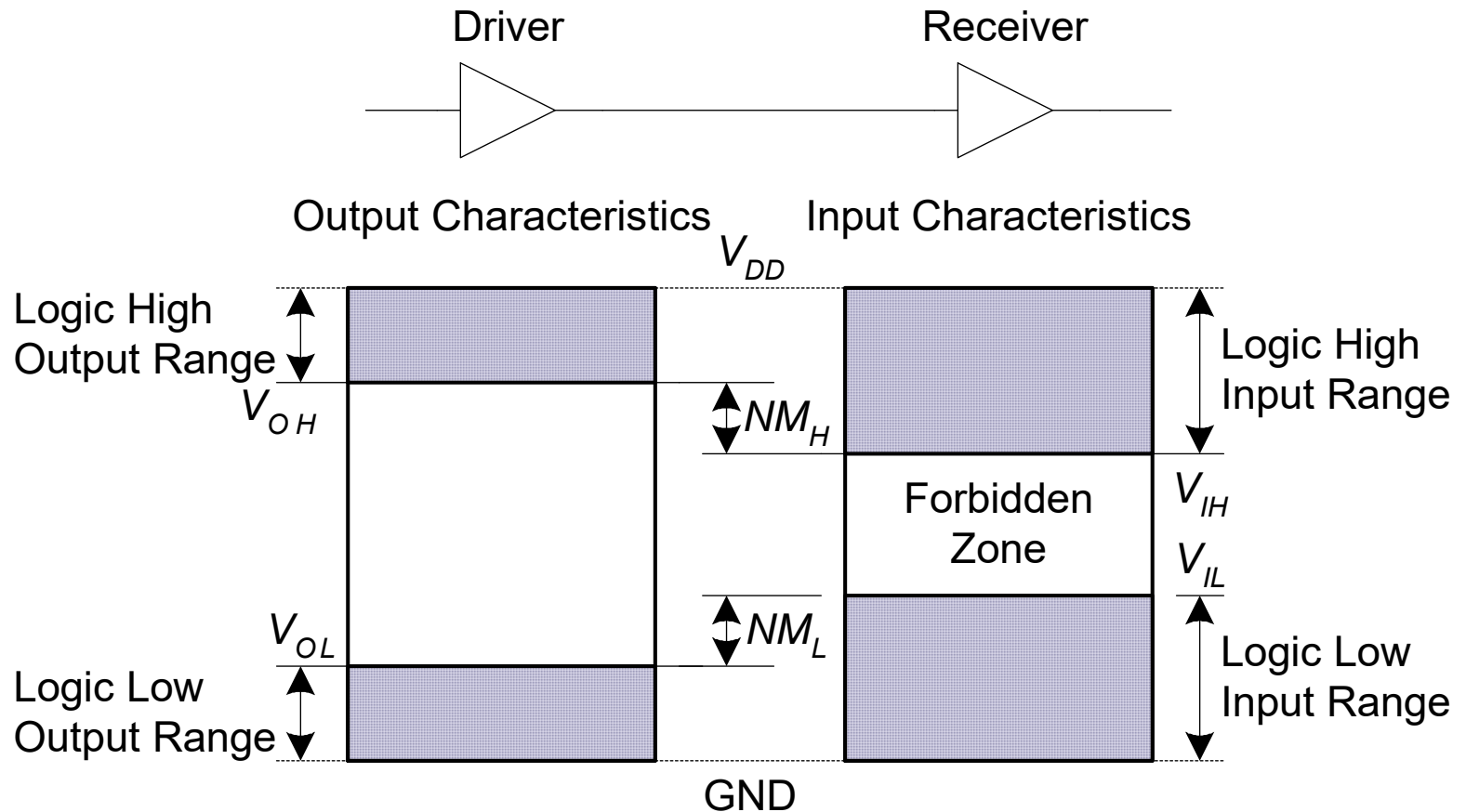
Logic Levels

- *Range* of voltages for 1 and 0
- Different ranges for inputs and outputs to allow for *noise*

Logic Levels



Noise Margins



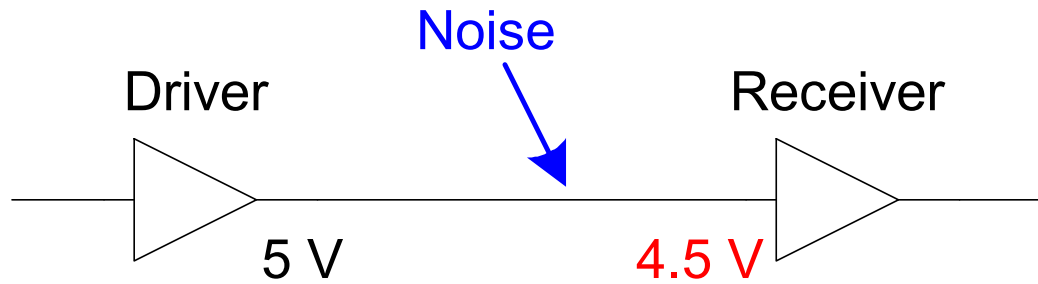
$$NM_H = V_{OH} - V_{IH}$$

$$NM_L = V_{IL} - V_{OL}$$

What is Noise?

What is Noise?

- **Anything that degrades the signal**
 - E.g., resistance, power supply noise, coupling to neighboring wires, etc.
- **Example:** a gate (driver) outputs 5 V but, because of resistance in a long wire, receiver gets 4.5 V



The Static Discipline

- With logically valid inputs, every circuit element must produce logically valid outputs
- Use limited ranges of voltages to represent discrete values

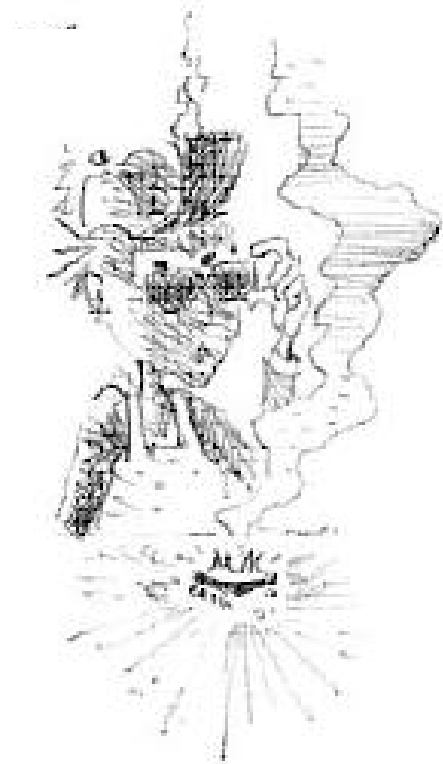
V_{DD} Scaling

- In 1970's and 1980's, $V_{DD} = 5\text{ V}$
- V_{DD} has dropped
 - Avoid frying tiny transistors
 - Save power
- 3.3 V, 2.5 V, 1.8 V, 1.5 V, 1.2 V, 1.0 V, ...
- Be careful connecting chips with different supply voltages

Chips operate because they contain magic smoke

Proof:

- if the magic smoke is let out, the chip stops working



Logic Family Examples

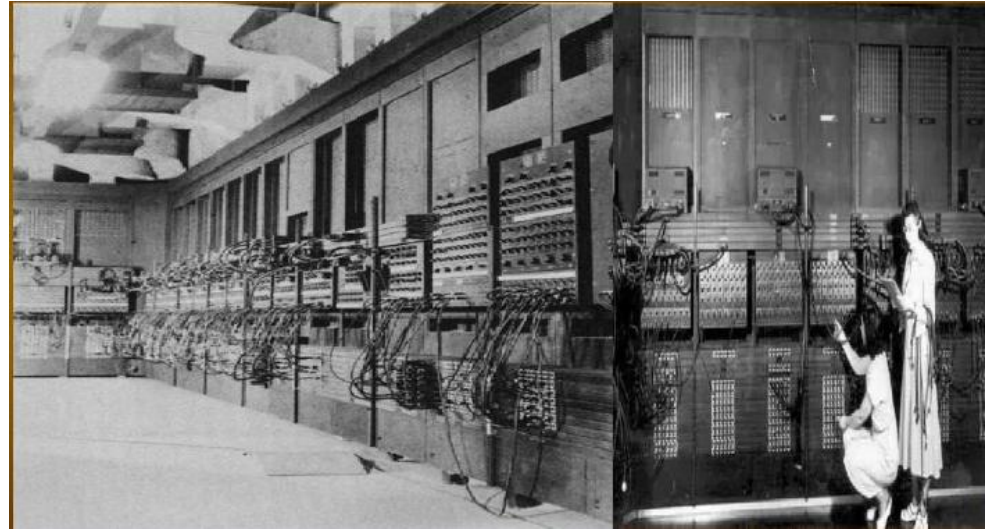
Logic Family	V_{DD}	V_{IL}	V_{IH}	V_{OL}	V_{OH}
TTL	5 (4.75 - 5.25)	0.8	2.0	0.4	2.4
CMOS	5 (4.5 - 6)	1.35	3.15	0.33	3.84
LVTTL	3.3 (3 - 3.6)	0.8	2.0	0.4	2.4
LVC MOS	3.3 (3 - 3.6)	0.9	1.8	0.36	2.7

Chapter 1 :: Topics

- Background
- The Game Plan
- The Art of Managing Complexity
- The Digital Abstraction
- Number Systems
- Logic Gates
- Logic Levels
- **CMOS Transistors**
- Power Consumption

Transistors

Electron tube (电子管)



A kind of amplifier, also used in TV and radio in early time.
The first computer in the world was designed with electron tube.
It used 18 million electron tubes, 30t weight, occupy 170m², 150kW energy consumption.

(ENIAC)

A lot of disadvantages

Transistors

transistor

in 1945, a Semiconductor research group was founded headed by **Shockley** in Bell lab.

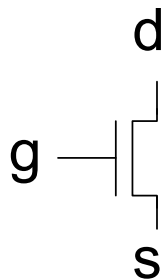
in 1947, transistor was invented. (by Shockley, Barding, Bratton)

in 1950, the first PN junction transistor was invented, which was a basic model used today.

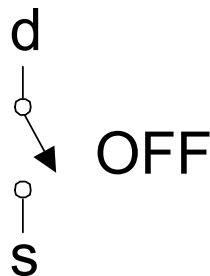
in 1956, Shockley, Barding, Bratton won Nobel Prize in Physics for inventing the transistor. (supreme honor)

Transistors

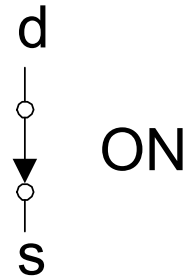
- Logic gates built from transistors
- 3-ported voltage-controlled switch
 - 2 ports connected depending on voltage of 3rd
 - d and s are connected (ON) when g is 1



$g = 0$

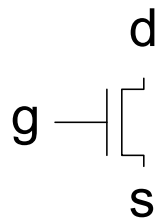


$g = 1$

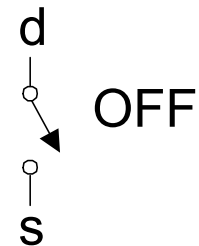


Transistor Function

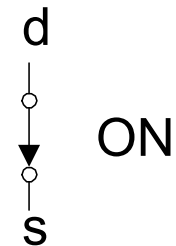
nMOS



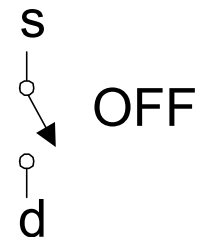
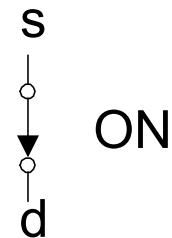
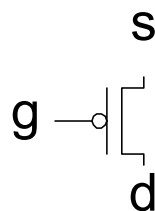
$g = 0$



$g = 1$

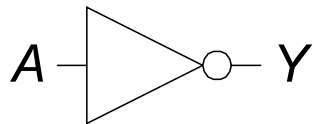


pMOS



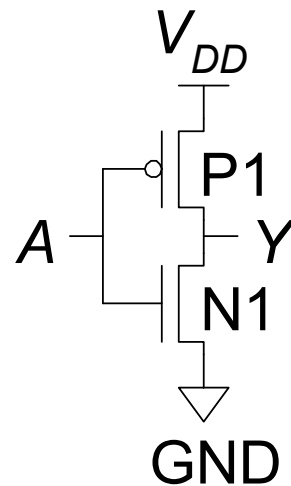
CMOS Gates: NOT Gate

NOT



$$Y = \overline{A}$$

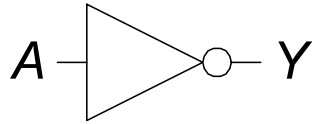
A	Y
0	1
1	0



A	P1	N1	Y
0			
1			

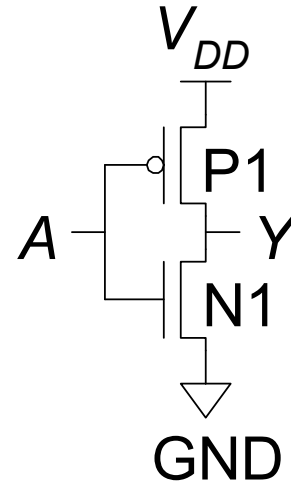
CMOS Gates: NOT Gate

NOT



$$Y = \overline{A}$$

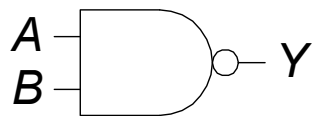
A	Y
0	1
1	0



A	P1	N1	Y
0	ON	OFF	1
1	OFF	ON	0

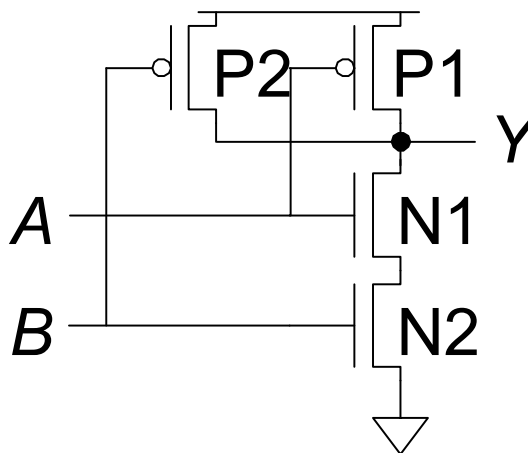
CMOS Gates: NAND Gate

NAND



$$Y = \overline{AB}$$

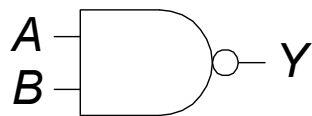
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



A	B	P1	P2	N1	N2	Y
0	0					
0	1					
1	0					
1	1					

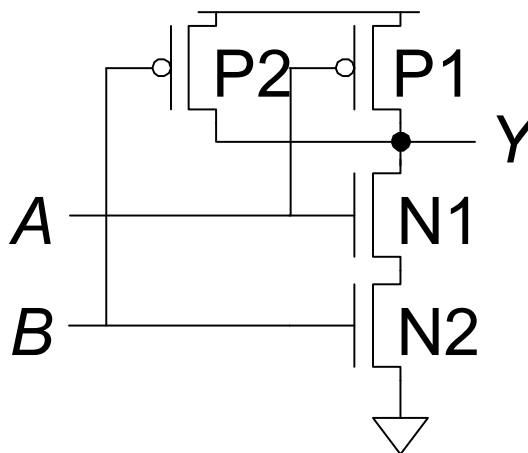
CMOS Gates: NAND Gate

NAND



$$Y = \overline{AB}$$

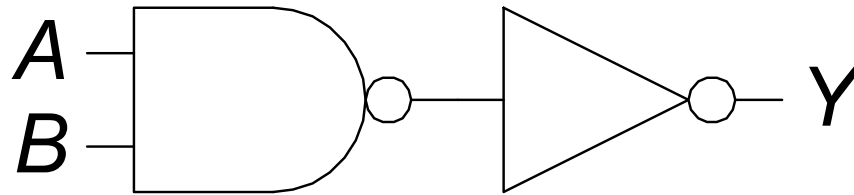
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



A	B	P1	P2	N1	N2	Y
0	0	ON	ON	OFF	OFF	1
0	1	ON	OFF	OFF	ON	1
1	0	OFF	ON	ON	OFF	1
1	1	OFF	OFF	ON	ON	0

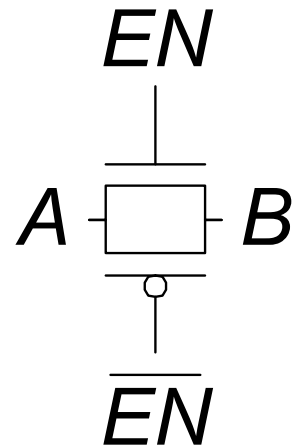
Other CMOS Gates

How do you build a two-input AND gate?



Transmission Gates

- nMOS pass 1's poorly
- pMOS pass 0's poorly
- Transmission gate is a better switch
 - passes both 0 and 1 well
- When $EN = 1$, the switch is ON:
 - A is connected to B
- When $EN = 0$, the switch is OFF:
 - A is not connected to B



Fairchild Semiconductor

- in 1956, Shockley, won Nobel Prize
- in 1956, 8 talents (诺依斯 (N. Noyce)、摩尔 (R. Moore)、布兰克 (J. Blank)、克莱尔 (E. Kliner)、赫尔尼 (J. Hoerni)、拉斯特 (J. Last)、罗伯茨 (S. Roberts) 和格里尼克 (V. Grinich)) join in Shockley's lab.
- in 1957, they founded fairchild semiconductor
- in 1958, 500,000 sales and 100 employees
- in 1959, invented integrated circuit (集成电路)
- in 1964, proposed Moore's Law
- in 1967, 200 million US dollars turnover

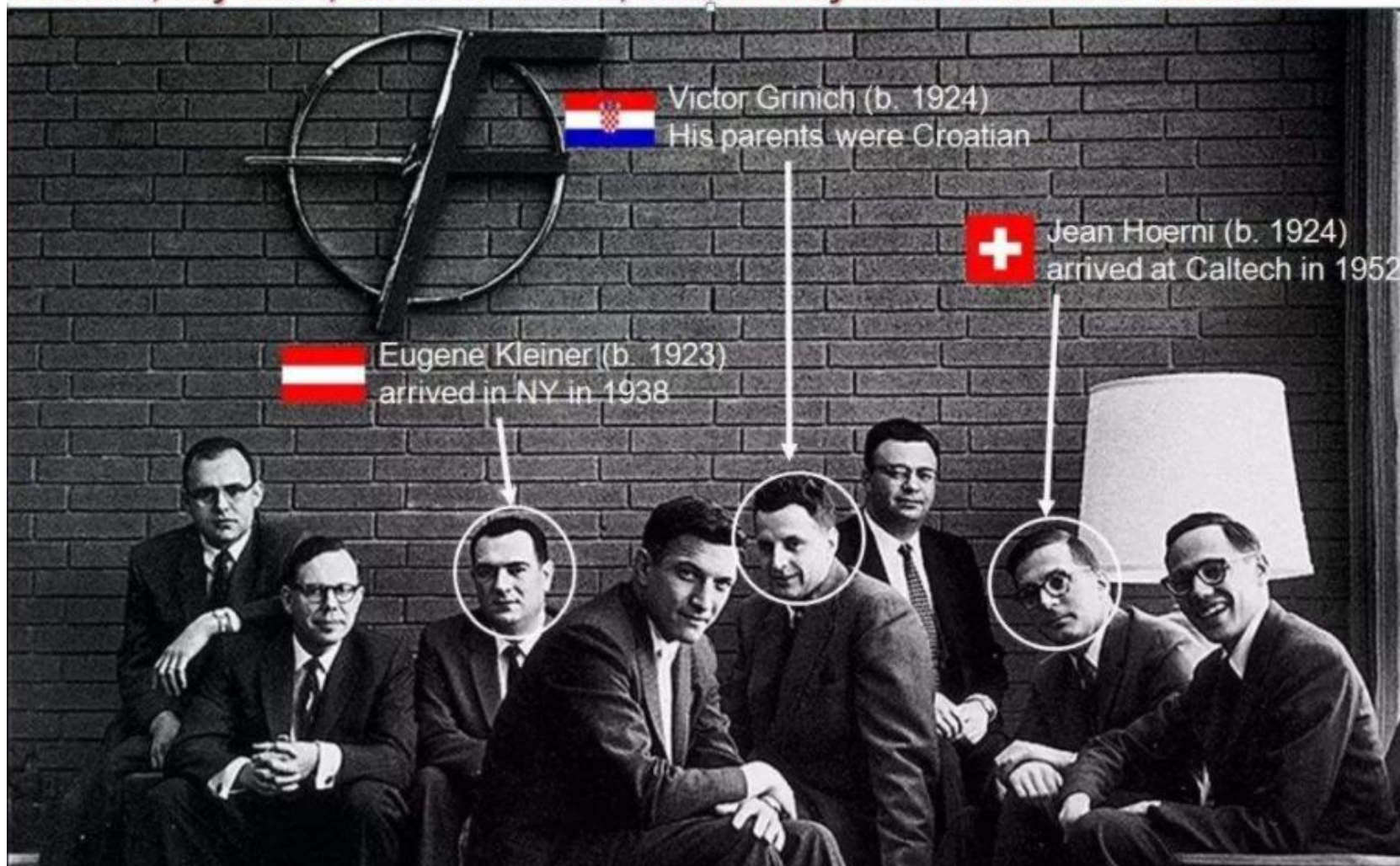
Fairchild Semiconductor

- Crisis
- bankrupt
- (arrogant)



- 1968, Intel (Noyce, Moore, Grove)
- 1968, AMD
- Fairchild Semiconductor Company is like a mature dandelion. As soon as you blow it, the seeds of this entrepreneurial spirit will float in the wind.

The eight Traitors: Julius Blank, Victor Grinich, Jean Hoerni, Eugene Kleiner, Jay Last, Gordon Moore, Robert Noyce and Sheldon Roberts.



Robert Noyce, 1927-1990

- Nicknamed “Mayor of Silicon Valley”
- Co-founded Fairchild Semiconductor in 1957
- Co-founded Intel in 1968
- Co-invented the integrated circuit

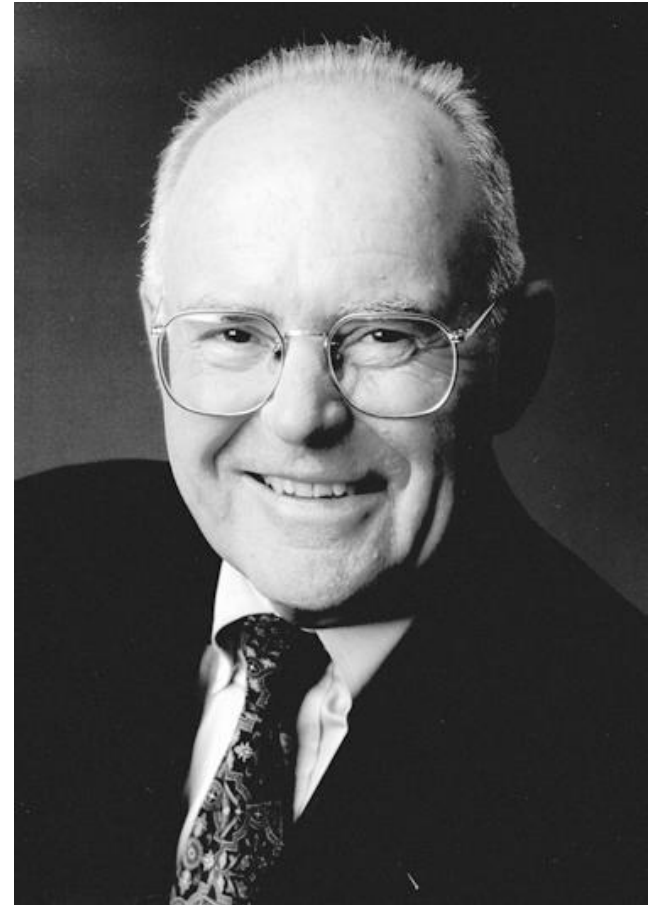


2011.12.12, 84th anniversary of Noyce, in memory of this great inventor, integrated circuit was used as the home page of Google.



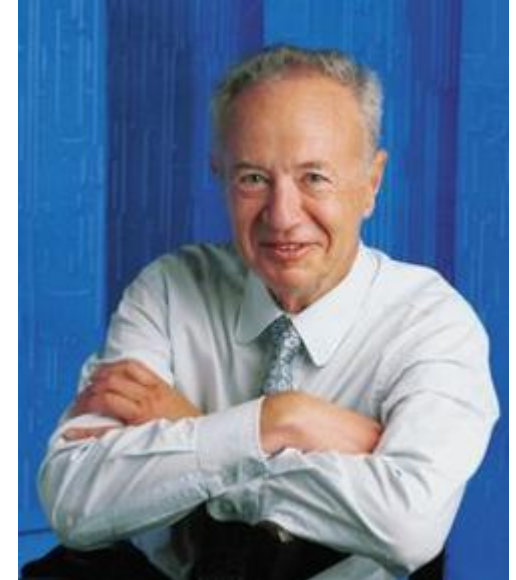
Gordon Moore, 1929-2023

- Cofounded Intel in 1968 with Robert Noyce.
- **Moore's Law:** number of transistors on a computer chip doubles every year (observed in 1965)
- Since 1975, transistor counts have doubled every two years.



Andy Grove 1936-2016

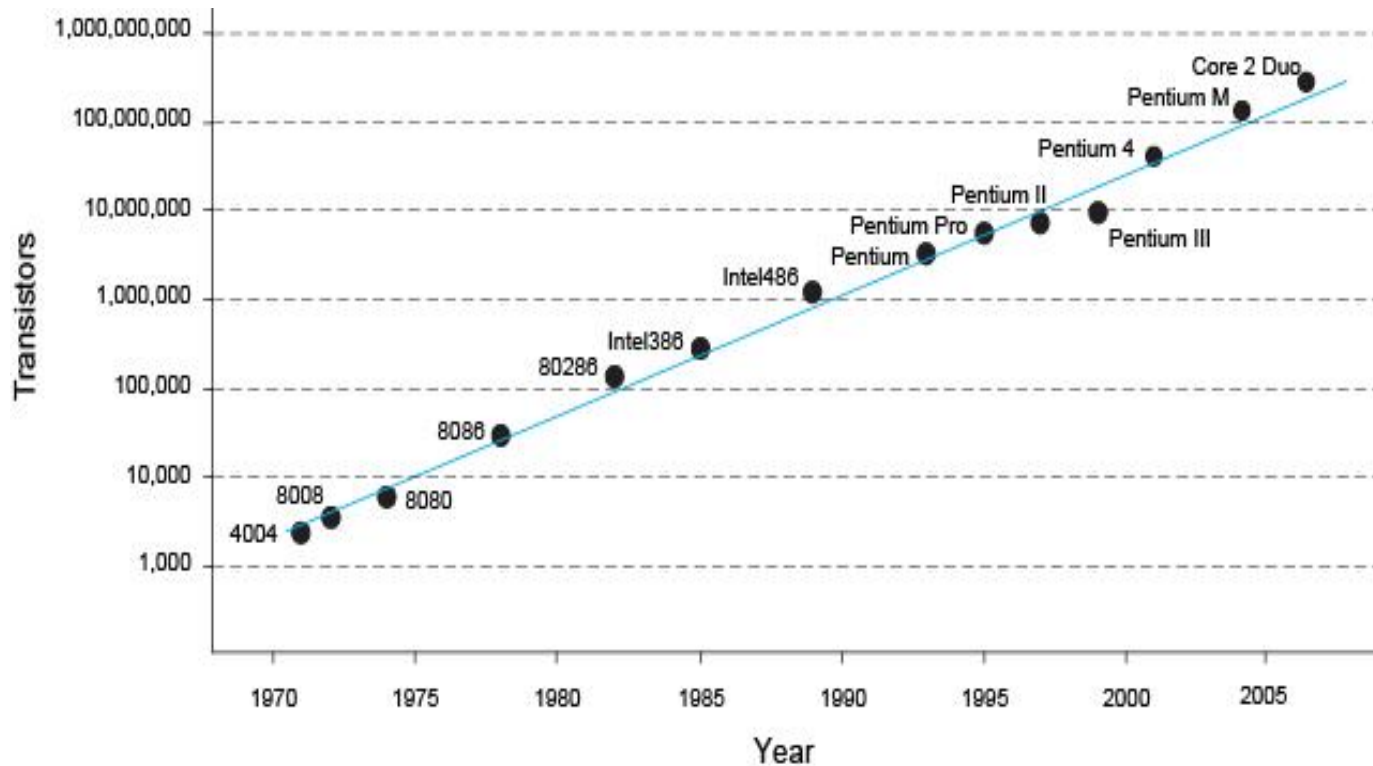
One of the greatest entrepreneurs
One of the greatest managers



He is the idol of Steve Jobs.

Wrote many books about management.

Moore's Law



- *“If the automobile had followed the same development cycle as the computer, a Rolls-Royce would today cost \$100, get one million miles to the gallon, and explode once a year . . .”*

– Robert Cringley

Moore

**Chemistry Bachelor's degree, UC Berkeley;
Physical chemistry Phd degree,
California Institute of Technology;**

Noyce

Phd in MIT;

Grove

Phd in UC Berkeley;

Chapter 1 :: Topics

- Background
- The Game Plan
- The Art of Managing Complexity
- The Digital Abstraction
- Number Systems
- Logic Gates
- Logic Levels
- CMOS Transistors
- **Power Consumption**

Power Consumption

- Power = Energy consumed per unit time
 - Dynamic power consumption
 - Static power consumption

Dynamic Power Consumption

- **Power to charge transistor gate capacitances**
 - Energy required to charge a capacitance, C , to V_{DD} is CV_{DD}^2
 - Circuit running at frequency f : transistors switch (from 1 to 0 or vice versa) at that frequency
 - Capacitor is charged $f/2$ times per second (discharging from 1 to 0 is free)
- Dynamic power consumption:

$$P_{dynamic} = \frac{1}{2}CV_{DD}^2f$$

Static Power Consumption

- Power consumed when no gates are switching
- Caused by the *quiescent supply current*, I_{DD} (also called the *leakage current*)
- Static power consumption:

$$P_{static} = I_{DD}V_{DD}$$

Power Consumption Example

- Estimate the power consumption of a wireless handheld computer
 - $V_{DD} = 1.2 \text{ V}$
 - $C = 20 \text{ nF}$
 - $f = 1 \text{ GHz}$
 - $I_{DD} = 20 \text{ mA}$

Power Consumption Example

- Estimate the power consumption of a wireless handheld computer
 - $V_{DD} = 1.2 \text{ V}$
 - $C = 20 \text{ nF}$
 - $f = 1 \text{ GHz}$
 - $I_{DD} = 20 \text{ mA}$

$$\begin{aligned} P &= \frac{1}{2} C V_{DD}^2 f + I_{DD} V_{DD} \\ &= \frac{1}{2} (20 \text{ nF}) (1.2 \text{ V})^2 (1 \text{ GHz}) + \\ &\quad (20 \text{ mA}) (1.2 \text{ V}) \\ &= \mathbf{14.4 \text{ W}} \end{aligned}$$

End